

実験ログ: プロシージャルモデリング (001~120)

2026-09-24 | 収録 34 件

Houdini 研究部で、hython で実際に測った実験の記録。数字はどれも測った値。

収録している実験

- Box を山にして細分割し、位置で色を付ける
- subdivide の回数で、形と数はどう変わるか
- mountain と subdivide は、順序を変えると何が変わるか
- mountain のノイズパラメータは、それぞれ何に効くのか
- basis (ノイズの種類) 14通りで、形はどこまで変わるか
- height は純粋な倍率か。gain と bias は何に効くか
- crease で角を残す — どれだけの重みが要するのか
- polyextrude と boolean — 硬い形を作る2つの道具
- 看板の文字を立体にする — Distance はそのまま奥行き。角の丸めは 0.08 で止まる
- ならしても体積はほとんど減らない — 強さ160でも 99.60%、縮みは 0.40%だけ
- 隙間 0.01 の継ぎ目は、Snap Distance が 0.01 になって初めて閉じる — 0.0095 では閉じない
- boolean は答えと6桁まで一致する — 差の正体は円ではなく多角形だった
- 立方体を割り続けると、体積は約32.76%に落ち着く — bilinear だけはちょうど1.000000のまま
- peak は式どおりに動く — 球は $((r+d)/r)^3$ 、立方体は $(1+2d/\sqrt{3})^3$ に6桁一致
- 面積の測りは、平らな面ならぴったり合う — 球の足りない分は分割の2乗で減る
- 押し出した体積は式どおり — 面積×距離、Inset を入れても四角錐台の式に6桁一致
- Rotate Order の xyz は「x から順に掛ける」 — 6通りで最大 0.725534 (約42.5°) ずれる
- resample の点の数は切り上げ — 「最大の長さ」は本当に最大だった
- spiral の長さは折れ線の式に6桁一致 — 1巻き50分割で 0.06% 短い
- attribrandomize の分布は式どおり — Scale Around Middle は標準偏差だった
- extractcentroid の Center of Mass は、閉じていれば体積・開いていれば面積の中心
- triangulate2d の三角形は $2n-2-h$ 枚 — 9通りすべて式どおり、面積は凸包と一致
- groupexpand の1段は「隣」の決め方しだい — 点と辺共有は菱形、面の既定は正方形
- copyxform は「1回分をくり返す」ではなく「値を i 倍して1回かける」 — 回転+移動は輪にならない
- pointjitter の Scale は「幅」 — 各軸 $-s/2 \sim +s/2$ の一様で、箱の中に散る
- vdbcombine の和・積・差は球2つの式に収束 — ボクセル 0.0125 で 0.08% 以内、どれも少なめ
- timeblend は速度が無ければ直線、あれば F^2 を誤差0でつなぐ — v は「1秒あたり」
- sweep の管は曲がり角で細る — 不足は $1-\cos(\text{角}/2)$ 、Stretch Around Turns で $A \times L$ に一致
- attribfill の到着時間は「網の辺をたどる道のり」 — 四角の網ではマンハッタン距離そのもの
- uvflatten は円筒をゆがみ0.0001%で開く — 半球は SCP 13%・ABF 10% ゆがむ

- ・ relax は点を $2 \times \text{pscale}$ 近くまで離す — 余裕があれば97%、詰め込みに近いと73~77%
- ・ revolve の面積・体積は k 角の式に6桁一致 — ただし閉じた断面を回すと体積がマイナス（裏返し）
- ・ 面に沿った距離は Surface が正解 — Edge と attribfill は辺の道のり、Heat は約1%短い
- ・ measurethickness は殻の厚みを6桁で返す — 板の縁ではぼかしで細く出る。box の Use Divisions は面にならない

Box を山にして細分割し、位置で色を付ける

2026-09-24 | Houdini 21.0.700 | /obj/box_mountain | ノード 5 / 接続 4

何を作ったか

プロシージャルモデリングの最小構成を確認するための実験。box SOP で立方体を作り、mountain SOP でノイズによる凹凸を与え、subdivide SOP で細分割し、color SOP で色を乗せた。最後に null ノードを OUT として置き、そこを出力点にしている。

Houdini の基本的な考え方である「ジオメトリを次のノードに流し込んで加工を積み重ねる」形をそのまま辿っている。各ノードは前のノードの出力だけを入力として受け取るため、途中のどのノードでも結果を確認できる。

ノードの接続



スクリプトが実際に構築したネットワーク。水色の点は表示フラグ。

ノード	タイプ	デフォルトから変更したパラメータ
box1	box	(変更なし)
mountain1	mountain::2.0	elementsize = 0.8 height = 0.35
subdivide1	subdivide	(変更なし)
color1	color	colortype = 1 ramp2cb = 1.0 ramp2cg = 1.0 ramp2cr = 1.0 ramp2pos = 1.0

ノード	タイプ	デフォルトから変更したパラメータ
OUT	null	(変更なし)

結果



Houdini

OpenGL ROP による Smooth Wire Shaded

表示。ワイヤーフレームが乗っているため、ネットワークが生成したトポロジが確認できる。画像内のウォーターマークは Apprentice ライセンスによるもの。

ジオメトリ

ポイント数	26
プリミティブ数	24
バウンディングボックス	[-0.4794, -0.4768, -0.4767] ~ [0.4932, 0.4959, 0.4959]
ポイント属性	P, Cd
プリミティブ属性	なし
ディテール属性	なし

わかったこと

- ・ box は 8 ポイント / 6 プリミティブ。subdivide を 1 回かけると 26 ポイント / 24 プリミティブになる。Catmull-Clark 細分割なので、回数を増やすと立方体はどんどん球に近づく（2 回で既に球状になり、元が箱だと分からなくなった）。

- ・ mountain SOP

は既存のポイントを法線方向にノイズで押し出すもので、ポイントを増やさない。つまり box のままかけても 8 点しか動かないため、凹凸のディテールを出すには先に subdivide などで分割数を上げる必要がある。

- ・ color SOP の colortype は数値メニュー（0=Constant, 1=Bounding Box, 2=Random, 3=Ramp from Attribute, 4=Random from Attribute）。文字列で指定すると TypeError になる。今回は 1

を使い、バウンディングボックス内の位置をそのまま RGB に割り当てた。

- ・ color SOP を通すとポイント属性に Cd が追加される（P のみ → P, Cd）。属性が増えたかどうかはジオメトリの統計で確認できる。

次にやること

- ・ subdivide の回数を変えて、ポイント数と見た目がどう変わるかを並べて比較する
- ・ mountain を subdivide の後ろに移動し、分割後に凹凸をかけた場合との違いを見る
- ・ grid から地形を作る流れに広げ、プロシージャルモデリングの定番パターンに入る

subdivide の回数で、形と数はどう変わるか

2026-09-24 | Houdini 21.0.700 | /obj/subdiv_compare | ノード 5 / 接続 4

何を作ったか

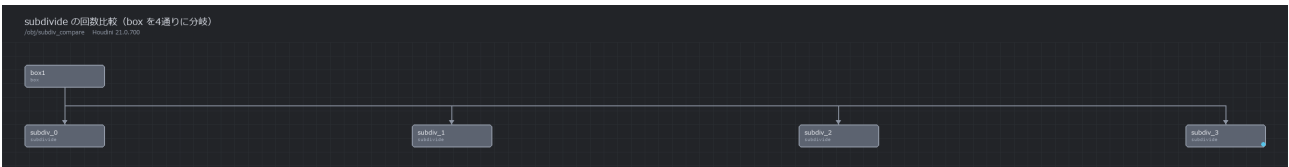
実験001で「subdivideの回数を増やすと立方体が球に近づく」ことに気づいたので、それを数値で確かめる実験。box SOP に subdivide SOP

を4通り（0回・1回・2回・3回）ぶら下げ、すべて同じカメラでレンダして並べた。

同じカメラで撮っているのが重要な点で、各変種を自動でフレームに収めてしまうと、大きさの違いが消えて比較にならない。画を見ると球が立方体よりはっきり小さいことが分かるが、これは目の錯覚ではなく実測でも縮んでいる。

さらに「なぜ縮むのか」「どこまで縮むのか」「1回目だけ縮まないのはなぜか」を追加検証で確定させた。

ノードの接続

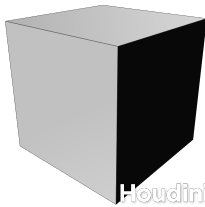


スクリプトが実際に構築したネットワーク。水色の点は表示フラグ。

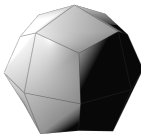
ノード	タイプ	デフォルトから変更したパラメータ
box1	box	(変更なし)
subdiv_0	subdivide	iterations = 0
subdiv_1	subdivide	(変更なし)
subdiv_2	subdivide	iterations = 2
subdiv_3	subdivide	iterations = 3

回数を変えた結果

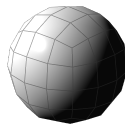
すべて同じカメラ・同じ表示モード（Smooth Wire Shaded）。元の立方体の輪郭に対して、分割を重ねるほど内側に縮んでいくのが分かる。



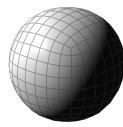
0回: 8点 / 6面 一辺
1.00000



1回: 26点 / 24面 一辺
1.00000



2回: 98点 / 96面 一辺
0.87847



3回: 386点 / 384面 一辺
0.84918

iterations	ポイント数	プリミティブ数	面数の倍率	一辺の長さ
0	8	6	—	1.00000
1	26	24	4.00	1.00000

iterations	ポイント数	プリミティブ数	面数の倍率	一辺の長さ
2	98	96	4.00	0.87847
3	386	384	4.00	0.84918
4	1538	1536	4.00	0.84192
5	6146	6144	4.00	0.84011
6	24578	24576	4.00	0.83966

アルゴリズム別の比較（iterations = 3 で固定）

subdivide SOP の algorithm パラメータには5つの選択肢がある。既定値は 2（OpenSubdiv Catmull-Clark）。同じ3回分割で結果がどう違うかを測った。

algorithm	ポイント数	プリミティブ数	一辺の長さ
Houdini Catmull-Clark	386	384	0.84918
Mantra-Compatible Catmull-Clark	386	384	0.83951
OpenSubdiv Catmull-Clark（既定）	386	384	0.84918
OpenSubdiv Loop	386	768	0.86932
OpenSubdiv Bilinear	386	384	1.00000

回数を変えたときの一辺（2つのアルゴリズムの比較）

OpenSubdiv Catmull-Clark は回数を増やすほど値が動くが、Mantra-Compatible は回数を変えても一定。これが何を意味するかは「わかったこと」の最後に書いた。

iterations	OpenSubdiv CC	Mantra-Compatible
1	1.00000	0.83951
2	0.87847	0.83951
3	0.84918	0.83951
4	0.84192	0.83951
5	0.84011	0.83951

わかったこと

- iterations = 0 は素通し。8点 / 6面のまま何も起きない。比較の基準として使える。
- プリミティブ数は1回ごとに正確に4倍になる（6 → 24 → 96 → 384 → 1536 → 6144 → 24576）。四角形1枚が4枚に割れるため。3回で64倍、6回で4096倍なので、回数を安易に増やすと破綻する。
- ポイント数は常に「プリミティブ数 + 2」になっている。これは偶然ではなくオイラーの多面体定理から導ける。閉じた四角形メッシュでは各辺を2つの面が共有するので $E = 2F$ 、これを $V - E + F = 2$ に代入すると $V = F + 2$ 。実測した7通りすべてで一致した。
- 1回目だけ一辺が 1.00000 のまま縮まない。検証したところ、+X方向の端を保持しているポイントは (0.5, 0, 0) の1点だけだった。これは元の +X 面の中心そのもの。Catmull-Clark は各面の中心に新しい点を作るため、1回目はその新点が元の面の上に乗る、バウンディングボックスが変わらない。2回目でその点も内側に引かれ 0.87847 に縮む。

- ・ 一辺の縮み方は等比的に収束する。差を取ると $0.02929 \rightarrow 0.00726 \rightarrow 0.00181 \rightarrow 0.00045$ で、毎回およそ $1/4$ （実測 $0.248 / 0.249 / 0.249$ ）。ここから外挿した極限值は約 0.83951 になる。

- ・ 縮みの原因は「分割」ではなく「平滑化」だと確定した。OpenSubdiv Bilinear（平滑化せず分割するだけの方式）で3回分割しても一辺は 1.00000 のまま変わらない。面数は同じ384なので、分割量は同じで平滑化の有無だけが違う。

- ・ OpenSubdiv Loop だけプリミティブ数が 768 になる（他は384）。Loop は三角形メッシュ向けの方式なので、四角形が三角形に割られて枚数が倍になる。四角形を保ちたい場面では選んではいけない。

- ・ Mantra-Compatible Catmull-Clark は回数を $1 \sim 5$ のどれにしても一辺が 0.83951 で一定だった。これは分割を繰り返して極限に近づけるのではなく、極限曲面上の位置を直接求めていることを示している。実際この値は、OpenSubdiv CC の収束列から外挿した極限值 0.83951 と一致した。つまり「無限回分割した先の形」を1回で出しているのがこのモード。

次にやること

- ・ crease（折り目）を使って角を保持できるか。立方体らしさを残したまま分割する方法があるはず

- ・ mountain と subdivide

の順序を入れ替えて、凹凸のディテールがどう変わるかを比較する（実験003）

- ・ polyextrude

など他のSOPでもポイント数・プリミティブ数の増え方を測り、重くなる原因を数で把握する

mountain と subdivide

は、順序を変えると何が変わるか

2026-09-24 | Houdini 21.0.700 | /obj/order_compare | ノード 14 / 接続 13

何を作ったか

実験001で「mountain はポイントを増やさないの、細かい凹凸を出すには先に分割数を上げる必要がある」と書いた。これは筋は通っているが、実際に確かめてはいなかった。そこで順序違いを並べて比較し、違いを数字で出す。

比較したのは6通り。ノイズなしの基準形、ノイズを先にかけたもの、分割を先にしたもの、分割をさらに細かくしたもの、ノイズの特徴サイズを小さくしたもの、そのうえで点数を増やしたもの。すべて同じカメラでレンダした。

この実験では測り方を一度間違えている。最初に選んだ指標では違いが正しく出ず、途中で別の指標に切り替えた。その経緯も含めて記録した。

ノードの接続



スクリプトが実際に構築したネットワーク。水色の点は表示フラグ。

ノード	タイプ	デフォルトから変更したパラメータ
box1	box	(変更なし)
A_mountain	mountain::2.0	elementsiz = 0.8 height = 0.35
A_subdiv	subdivide	iterations = 2
B_subdiv	subdivide	iterations = 2
B_mountain	mountain::2.0	elementsiz = 0.8 height = 0.35
C_subdiv	subdivide	iterations = 3
C_mountain	mountain::2.0	elementsiz = 0.8 height = 0.35
D_subdiv	subdivide	iterations = 3
D_mountain	mountain::2.0	elementsiz = 0.25 height = 0.35
E_subdiv	subdivide	iterations = 4
E_mountain	mountain::2.0	elementsiz = 0.25 height = 0.35
base_subdiv	subdivide	iterations = 2
base3_subdiv	subdivide	iterations = 3
base4_subdiv	subdivide	iterations = 4

6通りの結果

すべて同じカメラ・同じ表示モード。左から右へ、凹凸がどう変わるかを見てほしい。mountainのパラメータは height = 0.35 で固定し、elementsize（ノイズの特徴サイズ）だけを D と E で 0.8 から 0.25 に変えている。



Houdini

基準: 分割のみ、ノイズなし 98点 / 96面



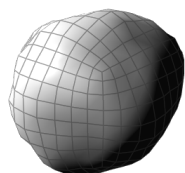
Houdini

A: mountain → subdiv(2) 98点 / 96面



Houdini

B: subdiv(2) → mountain 98点 / 96面



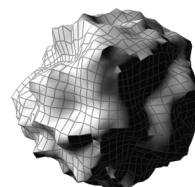
Houdini

C: subdiv(3) → mountain 386点 / 384面



Houdini

D: C と同じだが elementsize 0.25 / 386点



Houdini

E: D と同じノイズで 点数のみ増加 / 1538点

基準形からの点ごとのずれ

同じ分割回数ならポイントの順序も同じなので、ノイズなしの基準形と1点ずつ引き算できる。これが「ノイズがどれだけ形を動かしたか」を直接表す。A・B は2回分割の基準形と、C・D は3回分割の基準形と、E は4回分割の基準形と比べている。

変種	ポイント数	平均のずれ	最大のずれ
A: mountain → subdiv(2)	98	0.02069	0.03671
B: subdiv(2) → mountain	98	0.02479	0.08062
C: subdiv(3) → mountain	386	0.02504	0.08009
D: C + elementsize 0.25	386	0.03236	0.11985
E: D + 点数4倍	1538	0.03239	0.13173

最初を選んで失敗した指標（表面積）

凹凸が増えれば表面積も増えるはずだと考え、まず表面積を測った。ところが mountain を足すと表面積は減った。ノイズは外側にも内側にも押すため全体が縮み（一辺が 0.8785 から 0.8545

へ)、面積は大きさの2乗に比例するのでその影響が凹凸の効果を打ち消していた。一辺の2乗で割って正規化しても、値の大小が凹凸の量と対応しない。大きさと凹凸が混ざった指標だったということ。

変種	表面積	基準比	一辺	面積 / 一辺 ²
基準 (ノイズなし)	2.43164	1.000	0.8785	3.15076
A	2.30052	0.946	0.8545	3.15066
B	2.24947	0.925	0.8535	3.08797
C	2.16988	0.892	0.8314	3.13918
D	2.49922	1.028	0.9123	3.00282
E	2.66892	1.098	0.9278	3.10046

わかったこと

- ・順序を入れ替えてもポイント数とプリミティブ数は同じ (どちらも98点 / 96面)。つまりデータの重さは変わらない。変わるのは形だけ。
- ・最大のずれは A が 0.03671、B が 0.08062 で、B が約2.2倍大きい。同じ mountain の設定でも、分割したあとにかけた方が凹凸が残る。実験001で立てた見込みは正しかった。
- ・なぜそうなるかも測れた。box に mountain をかけた直後は、8点すべてが平均 0.03607 動いている。ところがそのあと subdivide をかけると、基準形からのずれは平均 0.02069・最大 0.03671 まで下がる。Catmull-Clark の平滑化は周囲の点を平均するので、せっかく動かした凹凸をなめしてしまう。
- ・C は B とほぼ同じずれだった (平均 0.02504 対 0.02479、最大 0.08009 対 0.08062)。点を4倍に増やしても凹凸の大きさは変わらない。点数はノイズの振幅を決めていない。
- ・振幅を決めていたのは elementsize だった。C と D は同じ386点だが、elementsize を 0.8 から 0.25 に下げると平均のずれが 0.02504 から 0.03236 に増え、画も明確に岩ようになった。
- ・E は D と同じノイズ設定で点数だけ4倍にしたもの。平均のずれは 0.03236 から 0.03239 でほぼ変わらない。一方で画は、同じ起伏をより滑らかな面で表現できている。つまり点数は「ノイズをどれだけ忠実に表現できるか (解像度)」を決めていて、「起伏の大きさ (振幅)」は決めていない。
- ・ただし最大のずれだけは点数とともに少し増える (0.11985 → 0.13173)。点が増えるとノイズのより極端な場所まで拾えるようになるため、振幅そのものが増えたわけではない。
- ・実用的な結論: 細かいディテールが欲しいときは、分割数を上げるだけでは足りない。elementsize も一緒に小さくする。そして mountain は分割のあとに置く。この3つが揃って初めて狙った凹凸になる。
- ・測り方についての反省。表面積のような「全体をひとつの数にまとめた量」は、大きさと形の変化が混ざってしまう。基準形と1点ずつ対応づけて引き算する方が、知りたいことを素直に測れた。比較実験では、まず基準形を作り、それと1対1で比べられる量を選ぶべき。

次にやること

- ・mountain の他のパラメータ (roughness, octaves) が振幅と細かさのどちらに効くのかを、同じやり方で切り分ける
- ・grid から地形を作り、平面の場合も同じ結論になるか確かめる (実験004)
- ・crease を使って、分割しても角を保持できるかを確かめる (実験002からの持ち越し)

mountain

のノイズパラメータは、それぞれ何に効くのか

2026-09-24 | Houdini 21.0.700 | /obj/mountain_params | ノード 15 / 接続 14

何を作ったか

実験003で「点数は解像度を決め、elementsize は振幅を決める」と分かった。では残りのノイズパラメータは何を変えているのか。同じ切り分け方で調べる。

最初に分かったのは名前の問題だった。パラメータ名は roughness や octaves ではなく rough / oct / lac。実際に mountain::2.0 が持っている名前を確認してから使わないと、そもそも動かない。

この実験でも測り方を追加している。003で使った「基準形からのずれ」だけでは、これらのパラメータをほとんど区別できなかった。振幅ではなく形の性質を変えるパラメータだったため、性質を測る指標を新しく用意した。

ノードの接続



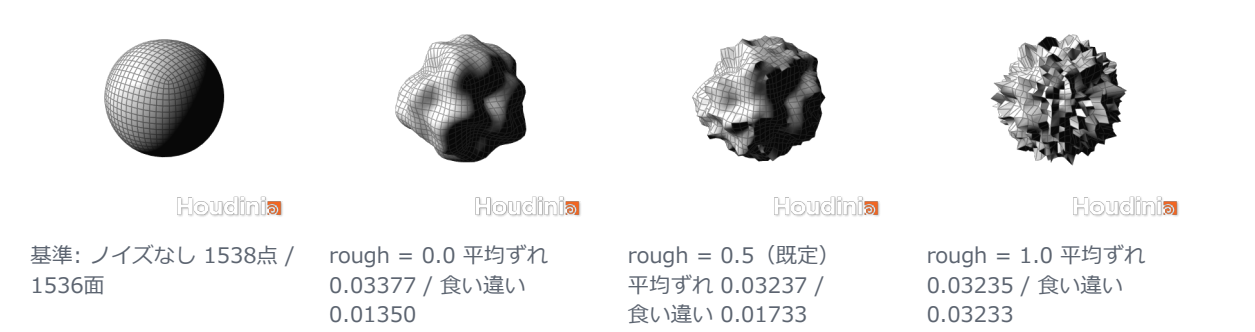
スクリプトが実際に構築したネットワーク。水色の点は表示フラグ。

ノード	タイプ	デフォルトから変更したパラメータ
box1	box	(変更なし)
base_subdiv	subdivide	iterations = 4
rough_00	mountain::2.0	elementsize = 0.25 height = 0.35 rough = 0.0
rough_05	mountain::2.0	elementsize = 0.25 height = 0.35
rough_10	mountain::2.0	elementsize = 0.25 height = 0.35 rough = 1.0
oct_1	mountain::2.0	elementsize = 0.25 height = 0.35 oct = 1
oct_3	mountain::2.0	elementsize = 0.25 height = 0.35 oct = 3
oct_8	mountain::2.0	elementsize = 0.25 height = 0.35
lac_15	mountain::2.0	elementsize = 0.25 height = 0.35 lac = 1.5
lac_20	mountain::2.0	elementsize = 0.25 height = 0.35 lac = 2.0
lac_40	mountain::2.0	elementsize = 0.25 height = 0.35 lac = 4.0

ノード	タイプ	デフォルトから変更したパラメータ
fractal_hmFT	mountain::2.0	elementsiz = 0.25 height = 0.35
fractal_fBm	mountain::2.0	elementsiz = 0.25 fractal = fBm height = 0.35
fractal_mfT	mountain::2.0	elementsiz = 0.25 fractal = mfT height = 0.35
fractal_none	mountain::2.0	elementsiz = 0.25 fractal = none height = 0.35

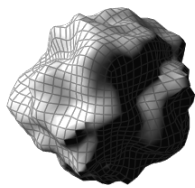
rough（粗さ） — 振幅は変えず、細かさだけを変える

固定条件はすべての変種で共通: subdivide 4回（1538点）、height 0.35、elementsiz 0.25。rough だけを動かしている。平均のずれはほぼ変わらないのに、見た目はなだらかな丘から棘だらけの形まで変わる。



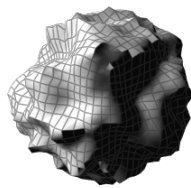
oct（オクタブ数） — 3 以上で効かなくなる

オクタブとは、細かさの違うノイズを重ねる層のこと。oct はその層の数。1 から 3 では効くが、3 から 8 ではほぼ変わらない（0.01732 → 0.01733）。点数が足りず、細かい層を表現できていないため。



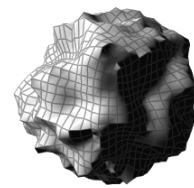
Houdini

oct = 1 食い違い 0.01519



Houdini

oct = 3 食い違い 0.01732



Houdini

oct = 8 (既定) 食い違い 0.01733

oct	平均のずれ	最大のずれ	隣との食い違い
1	0.03231	0.13417	0.01519
3	0.03250	0.13274	0.01732
8 (既定)	0.03237	0.13153	0.01733

予測の検証 — oct が効かないのは点数のせいかな

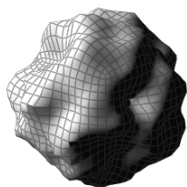
「oct が 3 以上で効かないのは点数が足りないからだ」という説明が正しいなら、点数を増やせば oct=8 が oct=3 を上回るはず。実際にそうだった。386点では oct=8 が oct=3 を下回っているが、6146点では上回っている。

注意: 食い違いの絶対値は点数が増えると小さくなる（隣同士の距離が縮まるので差も縮む）。この表は各行の中で比較だけが有効で、行をまたいで比べてはいけない。

subdivide	点数	oct = 1	oct = 3	oct = 8	8 と 3 の差
3回	386	0.02692	0.02868	0.02856	-0.00012
4回	1538	0.01519	0.01732	0.01733	+0.00001
5回	6146	0.00782	0.00941	0.00963	+0.00022

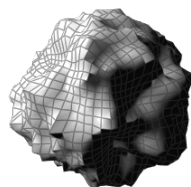
lac (ラキュナリティ) — オクターブ間の周波数の開き

重ねる層ごとに、どれだけ細かさを飛ばすかの倍率。既定は 1.841。大きくすると層の間隔が開いて細かい成分が強くなる。ただし lac = 1.5 では平均のずれも下がっており、振幅にも影響が出ている。



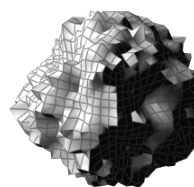
Houdini

lac = 1.5 平均 0.02625 / 食い違い 0.01259



Houdini

lac = 2.0 平均 0.03160 / 食い違い 0.01734



Houdini

lac = 4.0 平均 0.03169 / 食い違い 0.02218

lac	平均のずれ	最大のずれ	隣との食い違い
1.5	0.02625	0.13925	0.01259
2.0	0.03160	0.13528	0.01734
4.0	0.03169	0.13334	0.02218

fractal（フラクタルの種類）－ 差は小さい

層の重ね方の方式。4種類あるが、今回の条件では数値の差が小さかった。Terrain だけやや滑らか。既定は Hybrid Terrain。



Houdini

Hybrid Terrain（既定）
食い違い 0.01733



Houdini

Standard (fBm) 食い違い
0.01786



Houdini

Terrain 食い違い 0.01481



Houdini

None 食い違い 0.01350

fractal	平均のずれ	最大のずれ	隣との食い違い
Hybrid Terrain（既定）	0.03237	0.13153	0.01733
Standard (fBm)	0.03183	0.13207	0.01786
Terrain	0.03288	0.13504	0.01481
None	0.03377	0.14799	0.01350

わかったこと

- ・パラメータ名は rough / oct / lac。roughness や octaves という名前は存在せず、そのまま書くと何も設定されない。既定値は rough = 0.5、oct = 8、lac = 1.841、fractal = Hybrid Terrain、basis = Simplex。使う前に実際の名前を確認する必要がある。
- ・003で使った「基準形からのずれ」だけでは、これらのパラメータはほとんど区別できなかった。平均のずれはどれも 0.032～0.034 の範囲に収まる。振幅を決めているのは height と elementsize であり、今回のパラメータは振幅に効いていない。
- ・そこで「隣り合う点どうして、ずれの向きと量がどれだけ食い違うか」を測る指標を足した。なだらかな起伏なら隣の点も同じように動くので値は小さく、細かい凹凸なら大きくなる。基準形は原点中心なので、中心からの距離の差を符号付きのずれとして使っている（外に出たら正、へこんだら負）。これで完全に分離できた。
- ・rough が細かさに最も強く効く。0.0 から 1.0 で食い違いが 0.01350 から 0.03233 へ約2.4倍になる一方、平均のずれは 0.03377 から 0.03235 でほぼ変わらない。つまり rough は、同じ振幅のまま、なだらかな丘から棘だらけの形まで性質だけを変えるパラメータ。
- ・rough = 0.0 の数値は fractal = None と完全に一致した（平均 0.03377、最大 0.14799、食い違い 0.01350）。rough = 0 では追加の層が何も足さないの、層を重ねない単一ノイズに退化している。別の経路で同じ結果に到達したので、この解釈は確かだと言える。
- ・oct は 1 から 3 では効くが、3 から 8 ではほぼ変わらない（0.01732 → 0.01733、差は 0.00001）。点数が足りず細かい層を表現できていないため。この説明を予測に変えて検証したところ、点数を増やす

と oct = 8 が oct = 3 を上回り始めた（386点では -0.00012、1538点で +0.00001、6146点で +0.00022）。予測どおりだったので、この解釈も確かだと言える。

- ・実用的には、oct を上げるだけでは無駄になる。点数が足りていなければ、上げた層は形に現れない。003の「点数は解像度を決める」という結論と同じことが、別の角度から確認できた。

- ・lac も細かさに効く（1.5 → 4.0 で 0.01259 → 0.02218）。ただし lac = 1.5 では平均のずれも 0.02625 に下がっており、振幅にも影響している。rough のようにきれいに性質だけを変えるパラメータではない。

- ・fractal の4種類は、今回の条件では差が小さかった（食い違い 0.01350~0.01786）。地形向け的方式が揃っているので、平面の地形（実験005の予定）ではもっと差が出る可能性がある。

- ・指標の注意点。食い違いの絶対値は点数が増えると小さくなる。隣同士の距離が縮まれば、その間のずれの差も縮むため。したがって同じ点数の中でだけ比較できる。点数をまたいだ比較には、辺の長さで割るなどの正規化が必要になる。

- ・basis（ノイズの基礎となる種類）は14種類あった。既定は Simplex。Worley Cellular のようなセル状のものも含まれており、見た目が大きく変わるはずなので、別の実験にする価値がある。

次にやること

- ・basis を14種類すべて描き出して並べる。セル状ノイズは岩や結晶の見た目に直結するので、実用価値が高い
- ・grid から地形を作り、平面では fractal の種類の差が出るかを確かめる（実験005）
- ・食い違いの指標を辺の長さで正規化し、点数をまたいだ比較ができるようにする

basis（ノイズの種類） 14通りで、形はどこまで変わるか

2026-09-24 | Houdini 21.0.700 | /obj/noise_basis | ノード 16 / 接続 15

何を作ったか

004で basis に14種類あることが分かったので、同じ条件で全部描き出した。height 0.35、elementsize 0.25、subdivide 4回（1538点）で固定し、basis だけを変えている。

結果は他のパラメータと性質がまったく違った。rough は振幅を変えずに形の性質だけを変えたが、basis は振幅も性質も両方変える。同じ height を指定しても、種類によって実際のずれが4.5倍違う。

測る指標には004で作った「隣との食い違い」に加えて、外に出た点の割合を足した。ノイズが外向きに押すのか内向きに削るのかで、使いどころが変わるため。

ノードの接続



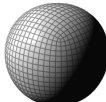
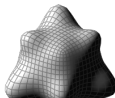
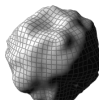
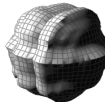
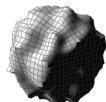
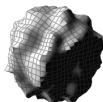
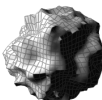
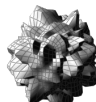
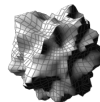
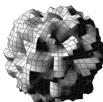
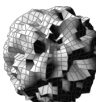
スクリプトが実際に構築したネットワーク。水色の点は表示フラグ。

ノード	タイプ	デフォルトから変更したパラメータ
box1	box	(変更なし)
base_subdiv	subdivide	iterations = 4
basis_sine	mountain::2.0	basis = sine elementsize = 0.25 height = 0.35
basis_perlin	mountain::2.0	basis = perlin elementsize = 0.25 height = 0.35
basis_pperlin	mountain::2.0	basis = pperlin elementsize = 0.25 height = 0.35
basis_simplex	mountain::2.0	elementsize = 0.25 height = 0.35
basis_sparse	mountain::2.0	basis = sparse elementsize = 0.25 height = 0.35
basis_flow	mountain::2.0	basis = flow elementsize = 0.25 height = 0.35
basis_pflow	mountain::2.0	basis = pflow elementsize = 0.25 height = 0.35
basis_worleyFA	mountain::2.0	basis = worleyFA elementsize = 0.25 height = 0.35
basis_worleyFB	mountain::2.0	basis = worleyFB elementsize = 0.25 height = 0.35

ノード	タイプ	デフォルトから変更したパラメータ
basis_mworleyFA	mountain::2.0	basis = mworleyFA elementsize = 0.25 height = 0.35
basis_mworleyFB	mountain::2.0	basis = mworleyFB elementsize = 0.25 height = 0.35
basis_cworleyFA	mountain::2.0	basis = cworleyFA elementsize = 0.25 height = 0.35
basis_cworleyFB	mountain::2.0	basis = cworleyFB elementsize = 0.25 height = 0.35
basis_alligator	mountain::2.0	basis = alligator elementsize = 0.25 height = 0.35

14種類すべて（細かさの小さい順）

すべて同じカメラ・同じ設定。左上のなめらかなものから、右下の硬く割れたものへ並んでいる。

					
Houdini	Houdini	Houdini	Houdini	Houdini	
基準: ノイズなし 1538点	Sinusoid 平均 0.14454 / 細かさ 0.00444	Sparse Convolution 平均 0.01985 / 細かさ 0.00965	Periodic Perlin Flow 平均 0.01551 / 細かさ 0.01005	Periodic Perlin 平均 0.01662 / 細かさ 0.01062	
					
Houdini	Houdini	Houdini	Houdini	Houdini	
Perlin 平均 0.02426 / 細かさ 0.01268	Perlin Flow 平均 0.02639 / 細かさ 0.01441	Simplex (既定) 平均 0.03237 / 細かさ 0.01733	Worley F2-F1 平均 0.14144 / 細かさ 0.01831	Worley F1 平均 0.08017 / 細かさ 0.02065	
					
Houdini	Houdini	Houdini	Houdini	Houdini	
Alligator 平均 0.12904 / 細かさ 0.02747	Manhattan F1 平均 0.08103 / 細かさ 0.02996	Chebyshev F2-F1 平均 0.11841 / 細かさ 0.03104	Chebyshev F1 平均 0.09937 / 細かさ 0.03193	Manhattan F2-F1 平均 0.11505 / 細かさ 0.03323	
basis	表示名	平均のずれ	最大のずれ	隣との食い違い	外向きの点の割合
sine	Sinusoid	0.14454	0.15748	0.00444	0.0%

basis	表示名	平均のずれ	最大のずれ	隣との食い違い	外向きの点の割合
sparse	Sparse Convolution	0.01985	0.08378	0.00965	32.9%
pflow	Periodic Perlin Flow	0.01551	0.05039	0.01005	51.2%
pperlin	Periodic Perlin	0.01662	0.05705	0.01062	58.8%
perlin	Perlin	0.02426	0.09921	0.01268	40.9%
flow	Perlin Flow	0.02639	0.10496	0.01441	49.9%
simplex	Simplex (既定)	0.03237	0.13153	0.01733	35.8%
worleyFB	Worley Cellular F2-F1	0.14144	0.17436	0.01831	0.0%
worleyFA	Worley Cellular F1	0.08017	0.17328	0.02065	5.4%
alligator	Alligator	0.12904	0.17456	0.02747	2.3%
mworleyFA	Manhattan Cellular F1	0.08103	0.16473	0.02996	91.1%
cworleyFB	Chebyshev Cellular F2-F1	0.11841	0.17439	0.03104	1.4%
cworleyFA	Chebyshev Cellular F1	0.09937	0.16540	0.03193	94.4%
mworleyFB	Manhattan Cellular F2-F1	0.11505	0.17052	0.03323	2.5%

わかったこと

- basis は振幅と性質の両方を変える。rough

が振幅を保ったまま性質だけを変えたのとは性格が違う。同じ height 0.35 でも、平均のずれは Periodic Perlin Flow の 0.01551 から Sinusoid の 0.14454 まで約9.3倍の開きがある。basis を変えたら height も再調整が必要。

- Sinusoid は例外的な挙動をする。平均のずれが 0.14454 と最大級に大きいのに、隣との食い違いは 0.00444 で全種類中もっとも小さい。外向きの点は 0.0%、つまり全部の点が内側に動いている。画を見ると、丸い4つの膨らみを持つ滑らかな形になっていた。規則的な波なので、細かい凹凸ではなく大きなうねりを作る。地形の大きな起伏には使えるが、ディテールには使えない。

- Perlin 系 (perlin / pperlin / flow / pflow) は外向きの点が 40~59% で、外と内にほぼ均等に動く。平均のずれも 0.0155~0.0264 と控えめ。自然な揺らぎを足したいときの素直な選択肢。

- 既定の Simplex は Perlin 系より振幅も細かさも大きい (平均 0.03237、細かさ 0.01733) が、セル系ほど極端ではない。中間的な位置にある。

- セル系 (Worley / Manhattan / Chebyshev / Alligator) は平均のずれが 0.08~0.14 と大きく、隣との食い違いも上位を占める。岩、割れ、結晶のような硬い見た目はここから作る。

- セル系の中で、外向きの点の割合が極端に分かれた。Manhattan F1 が 91.1%、Chebyshev F1 が 94.4% で、ほぼ全部の点が外に出る。一方 Worley F1 は 5.4%、Alligator は 2.3%、Chebyshev F2-F1 は 1.4% で、ほぼ全部が内側に削られる。

- ・画で確認すると理由が見えた。Worley F1

は、細胞の中心から遠いところが深く削られた侵食岩のような形になる。Manhattan F1 は距離の測り方が軸に沿っているため、軸に平行な面を持つ角ばった塊が外に突き出す形になった。同じ「F1」でも距離の定義が違うと、削る方向と形がまったく変わる。

・ F1 と F2-F1 の違いも外向き率に出る。Manhattan は F1 が 91.1% (外向き) なのに F2-F1 は 2.5% (内向き)。Chebyshev も F1 が 94.4%、F2-F1 が 1.4% で同じ傾向。F2-F1 は細胞の境界線を強調する計算なので、境界以外が下がる = 内向きになる。

・最大のずれはセル系でほぼ頭打ちになっている (0.164~0.175)。height 0.35 に対して 0.5 倍弱で止まっており、何らかの上限が効いている可能性がある。この確認は別の実験にする。

・外向きの点の割合という指標は安く作れて情報量が多かった。ノイズが押すのか削るのかは、実際に作りたいもの (突起か窪みか) に直結する。

次にやること

- ・セル系で最大のずれが 0.175 付近で止まる理由を確かめる。height を上げても止まるのかを測る
- ・grid の平面で同じ14種類を並べる。地形として見たときの向き不向きは球体とは違うはず
- ・外向き率を、突起を作りたいとき／窪みを作りたいときの選択指針としてまとめる

height は純粋な倍率か。gain と bias は何に効くか

2026-09-24 | Houdini 21.0.700 | /obj/height_shape | ノード 10 / 接続 9

何を作ったか

005でセル系の最大のずれが 0.164~0.175
に揃っていたので、何らかの上限が効いているのではないかと書いた。それを確かめる実験。height を 0.0875 から 1.4 まで16倍の範囲で変え、最大のずれが比例するかを見る。

結論を先に書くと、005の見立ては誤りだった。上限は存在しない。そして高い height で比率が崩れたのは Houdini の制限ではなく、私が使っている指標が壊れていたためだった。

あわせて未調査だった gain / bias / clipmin / clipmax も測った。bias は実用上かなり重要なパラメータだと分かった。

ノードの接続



スクリプトが実際に構築したネットワーク。水色の点は表示フラグ。

ノード	タイプ	デフォルトから変更したパラメータ
box1	box	(変更なし)
base_subdiv	subdivide	iterations = 4
h_simplex_035	mountain::2.0	elementsize = 0.25 height = 0.35
h_worleyFA_035	mountain::2.0	basis = worleyFA elementsize = 0.25 height = 0.35
gain_025	mountain::2.0	dogain = 1 elementsize = 0.25 gain = 0.25 height = 0.35
gain_05	mountain::2.0	dogain = 1 elementsize = 0.25 height = 0.35
gain_075	mountain::2.0	dogain = 1 elementsize = 0.25 gain = 0.75 height = 0.35
bias_025	mountain::2.0	bias = 0.25 dobias = 1 elementsize = 0.25 height = 0.35
bias_05	mountain::2.0	dobias = 1 elementsize = 0.25 height = 0.35
bias_075	mountain::2.0	bias = 0.75 dobias = 1 elementsize = 0.25 height = 0.35

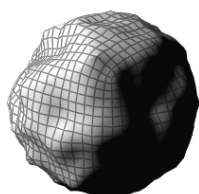
height を変えたときの最大のずれ

固定条件: subdivide 4回 (1538点)、elementsize 0.25。「最大 / height」が一定なら、height は単なる倍率ということになる。

basis	height	平均のずれ	最大のずれ	最大 / height	隣との食い違い
simplex	0.0875	0.00809	0.03289	0.37594	0.00433
simplex	0.1750	0.01619	0.06578	0.37590	0.00867
simplex	0.3500	0.03237	0.13153	0.37579	0.01733
simplex	0.7000	0.06473	0.26273	0.37532	0.03466
simplex	1.4000	0.12822	0.43122	0.30801	0.06868
worleyFA	0.0875	0.02005	0.04332	0.49513	0.00516
worleyFA	0.1750	0.04009	0.08665	0.49512	0.01033
worleyFA	0.3500	0.08017	0.17328	0.49508	0.02065
worleyFA	0.7000	0.16025	0.34636	0.49480	0.04125
worleyFA	1.4000	0.26687	0.42980	0.30700	0.07338

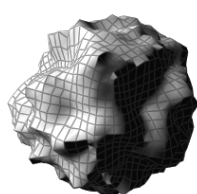
gain — 振幅の倍率。外向きと内向きの比率は変えない

gain は dogain を有効にしないと効かない（既定は無効）。gain = 0.5 のときの数値は、dogain を無効にした既定状態と完全に一致した。つまり 0.5 が中立値。



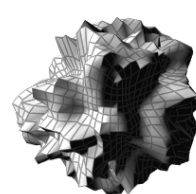
Houdini

gain 0.25 平均 0.01494 / 外向き 35.8%



Houdini

gain 0.50 (中立) 平均 0.03237 / 外向き 35.8%



Houdini

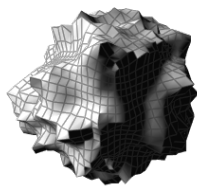
gain 0.75 平均 0.05519 / 外向き 35.8%

gain	平均のずれ	最大のずれ	隣との食い違い	外向きの点の割合
0.25	0.01494	0.07691	0.00816	35.8%
0.50 (中立)	0.03237	0.13153	0.01733	35.8%
0.75	0.05519	0.16402	0.02870	35.8%

bias — 外向きと内向きの比率を直接変える

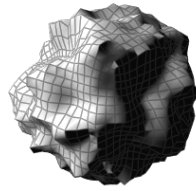
bias も dobias を有効にしないと効かない。0.5 が中立。0.25 にするとほぼ全部が内側に削られ、0.75 にするとほぼ全部が外側に膨らむ。005では basis

を選ぶことでしか得られなかった「突起か窪みか」の制御が、1つのパラメータで連続的にできる。



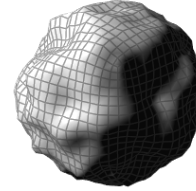
Houdini

bias 0.25 外向き 0.9% =
ほぼ全部が窪み



Houdini

bias 0.50 (中立) 外向き 35.8%



Houdini

bias 0.75 外向き 98.7% =
ほぼ全部が膨らみ

bias	平均のずれ	最大のずれ	隣との食い違い	外向きの点の割合
0.25	0.09756	0.16935	0.01614	0.9%
0.50 (中立)	0.03237	0.13153	0.01733	35.8%
0.75	0.07644	0.14054	0.01156	98.7%

わかったこと

- ・ height は純粋な倍率だった。「最大 / height」は 0.0875 から 0.7

まで8倍の範囲で小数点以下4桁まで一定 (simplex は 0.3759、worleyFA は 0.4951) 。 height を2倍にすれば、ずれもきっちり2倍になる。

- ・ 005で疑った上限は存在しなかった。セル系の最大のずれが揃って見えたのは、セル系ノイズのピーク値の分布が互いに似ているため。実際 005 のセル系7種類の「最大 / height」を計算すると 0.47~0.50 の範囲に収まる。一方 simplex は 0.376

で明確に違う。上限ではなく、ノイズの種類ごとの固有の比率だった。

- ・ ただし height = 1.4 では比率が両方とも 0.307 に落ちた。これは Houdini の制限ではなく、私の指標が壊れているため。基準形の半径は約 0.42 (subdivide 4回のバウンディングボックスの一辺 0.84192 の半分) で、そのとき測れた最大のずれ 0.4312 / 0.4298 はこれとほぼ一致している。中心を貫通するほど強く押し込まれると、「中心からの距離の差」という測り方では値が頭打ちになる。

- ・ つまりこの指標には適用範囲がある。ずれが対象の半径より十分小さいときにしか使えない。今回でいえば height 0.7 まで。005の観察を「上限がある」と解釈しかけたのは、この限界を意識していなかったため。指標を作ったら、それが壊れる条件も押さえておく必要がある。

- ・ clipmin / clipmax の既定は 0.0 / 1.0、dogain / dobias は既定で無効。つまり既定状態ではクリップもゲインもバイアスも効いていない。005までの測定に余計な処理は入っていなかったことが確認できた。

- ・ gain は振幅の倍率として働く。0.25 / 0.5 / 0.75 で平均のずれが 0.01494 / 0.03237 / 0.05519 と増える。一方で外向きの点の割合は 35.8% のまま完全に動かない。形の偏りには触れず、強さだけを変えるパラメータ。

- ・ gain = 0.5 の数値は、dogain を無効にした既定状態と完全に一致した (平均 0.03237、最大 0.13153、細かさ 0.01733) 。0.5 が中立値だと確認できた。

- ・ bias は外向きと内向きの比率を直接変える。0.25 で外向き 0.9% (ほぼ全部が内側に削られる) 、0.5 で 35.8%、0.75 で 98.7% (ほぼ全部が外側に膨らむ) 。画で見ると、0.25 は深くえぐられて鋭い稜線が残る侵食的な形、0.75 は膨らんで浅いへこみが残る塊になる。

- ・ただし bias を 0.5 から動かすと平均のずれも増える (0.03237 → 0.09756 / 0.07644)。分布が片側に寄ると、ゼロから離れた点が増えるため。純粋に形の偏りだけを変えるパラメータではなく、振幅も一緒に動く。

- ・ここまでで mountain のパラメータ地図がほぼ埋まった。振幅を決めるのは height (純粋な倍率) と gain (倍率)。細かさを決めるのは rough (最も強い)、lac、oct (点数が足りている範囲で)。外向きか内向きかを決めるのは bias (連続的) と basis (種類で決まる)。そして何を表現できるかの上限は点数が決める。

次にやること

- ・ grid の平面で同じ測定をする。球体では中心からの距離で測れたが、平面では別の基準が必要になるので、指標の作り直しも兼ねる
- ・ 指標を辺の長さで正規化し、点数をまたいだ比較ができるようにする
- ・ パラメータ調査はここで区切り、scatter と copy to points に進む

crease で角を残す — どれだけの重みが必要なのか

2026-09-24 | Houdini 21.0.700 | /obj/crease_test | ノード 15 / 接続 14

何を作ったか

実験002で「subdivide を重ねると立方体が球になる」と測った。3回分割すると一辺が 1.0 から 0.84918 まで縮む。crease（折り目）を付ければ角が保たれるはずだが、どれだけの重みが必要なのかは分からない。

理屈の上では「重み N なら N 回分の分割まで角が保たれる」はず。3回分割するなら重み3で完全に元の立方体のままになる、という予測を立てて確かめた。

結果は予測どおりだった。ただしこの実験では、2つの指標が違う答えを出すという厄介な状況にも当たった。

ノードの接続

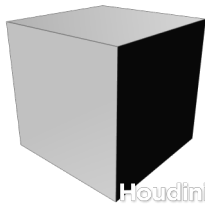


スクリプトが実際に構築したネットワーク。水色の点は表示フラグ。

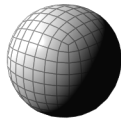
ノード	タイプ	デフォルトから変更したパラメータ
box1	box	(変更なし)
crease_0_0	crease	crease = 0.0 op = 1
sub_0_0	subdivide	iterations = 3
crease_0_5	crease	crease = 0.5 op = 1
sub_0_5	subdivide	iterations = 3
crease_1_0	crease	op = 1
sub_1_0	subdivide	iterations = 3
crease_2_0	crease	crease = 2.0 op = 1
sub_2_0	subdivide	iterations = 3
crease_3_0	crease	crease = 3.0 op = 1
sub_3_0	subdivide	iterations = 3
crease_4_0	crease	crease = 4.0 op = 1
sub_4_0	subdivide	iterations = 3
crease_10_0	crease	crease = 10.0 op = 1
sub_10_0	subdivide	iterations = 3

重みを変えて3回分割する

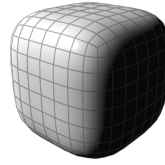
すべて同じカメラ。点数・面数はどの重みでも 386点 / 384面で変わらない。crease は形の計算方法を変えるだけで、ジオメトリを増やさない。



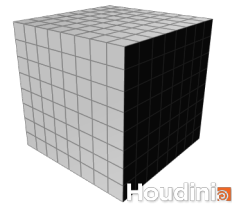
分割前の box 一辺
1.00000 / 折れ角 90.00度



重み 0.0 (crease なし)
一辺 0.84918 / 折れ角
13.93度



重み 1.0 一辺 1.00000 /
折れ角 28.07度

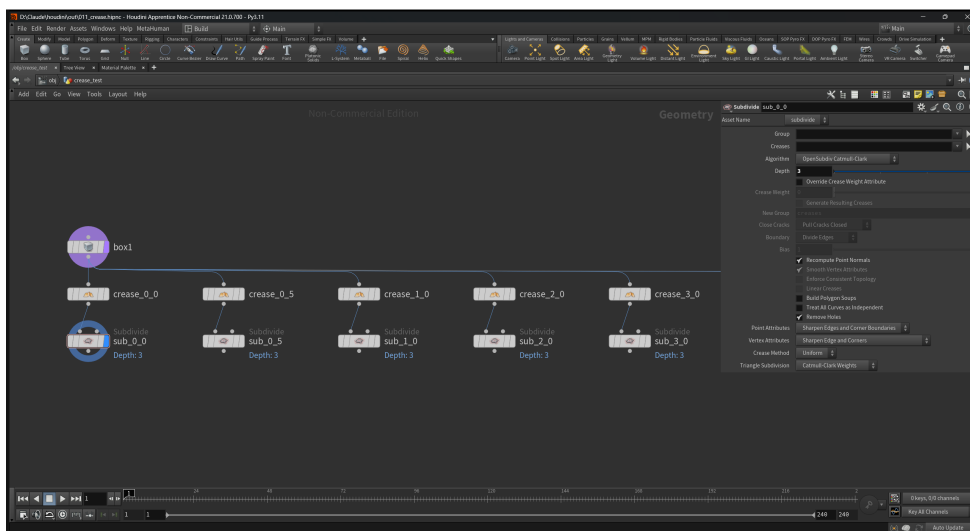


重み 3.0 一辺 1.00000 /
折れ角 90.00度

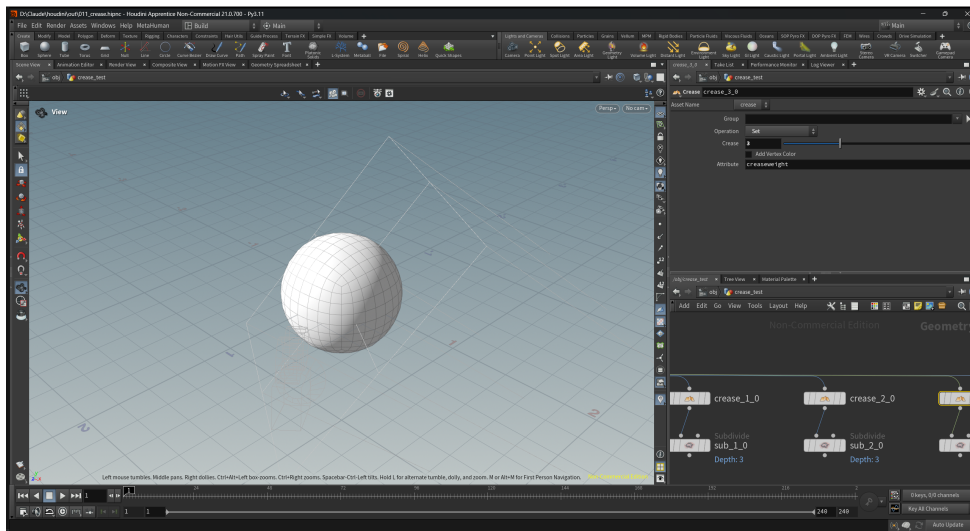
crease の重み	点数	面数	一辺の長さ	最大の折れ角
0.0 (なし)	386	384	0.84918	13.93度
0.5	386	384	0.92459	21.45度
1.0	386	384	1.00000	28.07度
2.0	386	384	1.00000	53.13度
3.0	386	384	1.00000	90.00度
4.0	386	384	1.00000	90.00度
10.0	386	384	1.00000	90.00度
(参考) 分割前の box	8	6	1.00000	90.00度

実際の Houdini の画面

crease SOP で折り目の重みを付け、その下流の subdivide がそれを読む。crease SOP の Crease Attribute の既定は creaseweight で、subdivide はこの名前の属性を自動で見ると。



box から crease と subdivide の組を7通り並べた



crease SOP のパラメータ。Operation を Set にして Crease に重みを入れる

わかったこと

- ・予測は当たった。3回分割する場合、重み 3.0 で折れ角が 90.00度 になり、分割前の box とまったく同じ角の鋭さになる。「重み N なら N 回分の分割まで角が保たれる」という理解でよい。
- ・重みを 3 より上げて何も変わらない (4.0 も 10.0 も 90.00度 のまま)。分割回数を超える重みは無駄。逆に言えば、分割回数を後から増やすつもりなら重みも合わせて上げる必要がある。
- ・点数・面数はどの重みでも 386点 / 384面で変わらなかった。crease は形の計算方法を変えるだけで、ジオメトリを増やさない。角を残すためにポリゴンを足す必要はない。
- ・2つの指標が違う答えを出した。一辺の長さは重み 1.0 で 1.00000 に達し、そこから変わらない。この指標だけを見ていたら「重み1で十分」と判断してしまう。しかし折れ角は 28.07度 で、まだ元の 90度 には程遠い。画を見ると角がはっきり丸い。
- ・つまり一辺の長さは「どこまで縮んだか」しか測っておらず、「角が鋭いかどうか」は測れていない。重み1で外形は元に戻るが、角は丸いまま。何を知りたいかによって、見るべき指標が変わる。
- ・折れ角の指標は、辺を共有する2つの面の法線のなす角の最大値として作った。角が残っていれば大きく、丸まれば小さくなる。分割前の box で 90.00度 と出るので、基準としても正しく働いている。
- ・重み 0.5 のとき一辺は 0.92459 だった。これは crease なし (0.84918) と重み1.0 (1.00000) のちょうど中間で、小数点以下5桁まで一致する。小数の重みは2つの結果を線形に混ぜている。
- ・ただし折れ角のほうは線形ではない。0.0 / 0.5 / 1.0 で 13.93 / 21.45 / 28.07度 となり、中間値の 21.00度 とはわずかにずれる。位置は線形に混ざるが、角度はそうならない。
- ・crease SOP の Operation の既定は「Add To Existing Value」で、既にある値に足し込む。重みを決め打ちしたいなら「Set」にする。既定のまま何度も通すと重みが積み上がる。
- ・crease SOP が書き込む属性名の既定は creaseweight で、subdivide はこの名前を自動で見る。名前を変えたい場合は subdivide 側の Creases にも同じ名前を指定する必要がある。

次にやること

- ・一部の辺だけに crease を付けて、角を残す場所と丸める場所を作り分ける
- ・polyextrude で作った形に crease を付け、建物のような硬い形を作る

- ・折れ角の指標を、面ごとではなく辺ごとに出して分布を見る（どの辺が丸まっているかを特定する）

polyextrude と boolean — 硬い形を作る2つの道具

2026-09-24 | Houdini 21.0.700 | /obj/extrude_boolean | ノード 15 / 接続 20

何を作ったか

ここまではノイズで有機的な形を作ってきた。建物や機械のような硬い形は別の道具で作る。面を押し出す polyextrude と、立体どうしを足し引きする boolean。

押し出しは距離が指定どおりになるのか、点数はどう増えるのか。boolean は演算ごとに何が変わるのか。そして boolean が正しく計算できているかを、体積の関係式で検算した。

ノードの接続



スクリプトが実際に構築したネットワーク。水色の点は表示フラグ。

ノード	タイプ	デフォルトから変更したパラメータ
grid	grid	cols = 5 rows = 5
ext_0_0	polyextrude::2.0	thicknessramp1interp = 2 thicknessramp1value = 1.0 thicknessramp2interp = 2 thicknessramp2pos = 1.0 thicknessramp2value = 1.0 twistramp1interp = 2 twistramp1value = 0.5 twistramp2interp = 2 twistramp2pos = 1.0 twistramp2value = 0.5
ext_0_5	polyextrude::2.0	dist = 0.5 thicknessramp1interp = 2 thicknessramp1value = 1.0 thicknessramp2interp = 2 thicknessramp2pos = 1.0 thicknessramp2value = 1.0 twistramp1interp = 2 twistramp1value = 0.5 twistramp2interp = 2 twistramp2pos = 1.0 twistramp2value = 0.5
ext_1_0	polyextrude::2.0	dist = 1.0 thicknessramp1interp = 2 thicknessramp1value = 1.0 thicknessramp2interp = 2 thicknessramp2pos = 1.0 thicknessramp2value = 1.0 twistramp1interp = 2 twistramp1value = 0.5 twistramp2interp = 2 twistramp2pos = 1.0 twistramp2value = 0.5

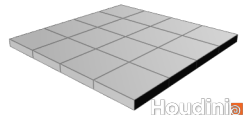
ノード	タイプ	デフォルトから変更したパラメータ
ext_2_0	polyextrude::2.0	dist = 2.0 thicknessramp1interp = 2 thicknessramp1value = 1.0 thicknessramp2interp = 2 thicknessramp2pos = 1.0 thicknessramp2value = 1.0 twistramp1interp = 2 twistramp1value = 0.5 twistramp2interp = 2 twistramp2pos = 1.0 twistramp2value = 0.5
bool_box	box	scale = 2.0
bool_sphere	sphere	cols = 30 radx = 1.35 radx = 1.35 radz = 1.35 rows = 30 type = 1
bool_union	boolean::2.0	(変更なし)
bool_intersect	boolean::2.0	booleanop = 1
bool_subtract	boolean::2.0	booleanop = 2
bool_shatter	boolean::2.0	booleanop = 3
bool_custom	boolean::2.0	booleanop = 4
bool_seam	boolean::2.0	booleanop = 6
bool_detect	boolean::2.0	booleanop = 8
bool_resolve	boolean::2.0	booleanop = 9

polyextrude の距離

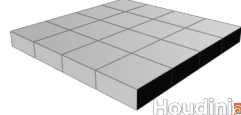
10×10 の grid (5分割 = 25点 / 16面) を押し出した。距離を 0 にすると何も起きず、0 より大きければ点数・面数は距離によらず一定になる。



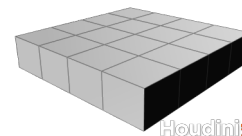
距離 0.0 25点 / 16面



距離 0.5 41点 / 32面



距離 1.0 41点 / 32面



距離 2.0 41点 / 32面

距離	点数	面数	できた高さ	指定との差
0.0	25	16	0.00000	0.00000
0.5	41	32	0.50000	0.00000
1.0	41	32	1.00000	0.00000
2.0	41	32	2.00000	0.00000

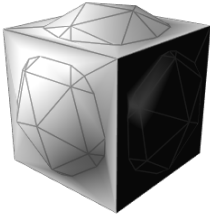
inset (内側への縮み)

距離 1.0 で固定して inset だけ変えた。上面がどれだけ小さくなるかを測っている。元の一辺は 10.0。

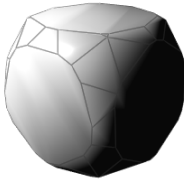
inset	点数	面数	上面の一辺	元との差
0.0	41	32	10.00000	0.0
0.3	41	32	9.40000	0.6 = 0.3 × 2
0.8	41	32	8.40000	1.6 = 0.8 × 2

boolean の演算ごとの結果

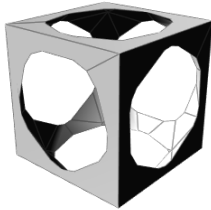
2×2×2 の box（体積 8.00000）と、半径 1.35 のポリゴン球（体積 9.00180）を組み合わせた。



Union（和集合） 体積 9.91613



Intersect（積集合） 体積 7.08567



Subtract（差集合） 体積 0.91433

演算	点数	面数	体積
Union（和集合）	130	120	9.91613
Intersect（積集合）	96	60	7.08567
Subtract（差集合）	104	60	0.91433
Shatter（砕く）	200	120	8.00000
Custom	200	120	8.00000
Seam（境界線）	88	6	0.00000
Detect（検出）	8	6	8.00000
Resolve（重なり解消）	152	496	2.91612

boolean が正しく計算できているかの検算

集合の大きさには必ず成り立つ関係がある。これを体積で確かめれば、boolean が正しく動いているかを数値で検証できる。

確かめた式	左辺	右辺	差
和集合 + 積集合 = box + 球	17.00180	17.00180	0.0000049
差集合 = box - 積集合	0.91433	0.91433	0.0000000

わかったこと

・ polyextrude の距離は指定どおりに出る。0.5 / 1.0 / 2.0 を指定して、できた高さは 0.50000 / 1.00000 / 2.00000 で差はゼロ。これは当たり前に見えて重要で、ノイズ系のパラメータ（height など）が種類によって実際のずれが9倍も違ったのとは対照的。押し出しは寸法をそのまま指定できる。

- ・点数・面数は距離によらず一定（41点 / 32面）。距離 0 のときだけ何も起きない（25点 / 16面のまま）。形を変えても重さは変わらない。
- ・元の 25点 / 16面 が 41点 / 32面 になる内訳も説明できる。5×5 の grid は境界に 16点あり、押し出すとその 16点が複製されて $25 + 16 = 41$ 点。面は上面 16 + 側面 16 = 32面。
- ・inset は各辺から内側に引っ込む距離。0.3 にすると一辺が両側から 0.3 ずつで 0.6 縮み、10.0 → 9.40000 になった。0.8 なら 1.6 縮んで 8.40000。指定した値がそのまま距離として効く。
- ・パラメータ名は dist と inset。分割数は divisions ではなく divs。ここでも一般的な名前ではなく実際の名前を確認する必要があった。
- ・boolean には8つの演算がある。よく使う Union / Intersect / Subtract のほかに、Shatter（Aを砕いて全部の破片を残す）、Seam（境界線だけを出す）、Detect（検出だけして形は変えない）、Resolve（自分自身の重なりを解消する）がある。
- ・Shatter の体積は 8.000000 で box と完全に一致した。砕いても全部の破片が残るので体積は変わらない。破壊エフェクトの下準備がこれ。
- ・Seam は面数 6 のまま体積 0 になる。境界線だけを取り出す演算なので立体にならない。
- ・体積の関係式で検算できた。集合には「和集合 + 積集合 = 元の2つの合計」という関係が必ず成り立つ。実測すると 17.00180 対 17.00180 で差は 0.0000049。また「差集合 = box - 積集合」も 0.91433 対 0.91433 で差は完全にゼロ。boolean が正しく計算できていることを数値で確認できた。
- ・この検算は、形が正しいかどうかを目で見ずに確かめられる点で価値がある。複雑な boolean の結果が正しいか不安なときは、体積の関係を見れば判断できる。
- ・sphere SOP で rows / cols を 30 に設定したが、面数は 80 のままだった。ポリゴン球は rows / cols ではなく別の分割方法（多面体の細分）を使っている。分割数を上げたいなら別のパラメータを探す必要がある。
- ・その粗い球（80面）の体積は 9.00180 だった。半径 1.35 の真球なら 10.30575 なので、約 13% 小さい。ポリゴンで近似した球は必ず内側に寄るので、体積を当てにするなら分割数を上げる必要がある。

次にやること

- ・polyextrude を繰り返して階層のある建物を作る。for-each との組み合わせ
- ・sphere の分割数を上げる方法を調べ、体積が真球にどう近づくかを測る
- ・エフェクトに入るための下準備（複数フレームの書き出し、シミュレーションのキャッシュ）

看板の文字を立体にする — Distance

はそのまま奥行き。角の丸めは 0.08 で止まる

2026-09-24

何を作ったか

font で文字の形を作り、polyextrude で厚みを付け、polybevel で角を丸める。手順ページに書く前に、3つのつまみの意味を数字で確かめた。

細かさ。font の Level of Detail は、点の数にほぼ比例する。文字「SP」・Font Size 1 で、LOD 0.2 / 0.5 / 1 / 2 / 4 のとき 222 / 518 / 962 / 1,850 / 3,626点。倍にすると点も倍になる。幅 1.1220・高さ 0.7390 は、どの LOD

でもまったく同じで、形の大きさは変わらない。面は2枚（文字1つが1面）のまま。

厚み。polyextrude の Distance は、そのまま奥行きになった。0.05 / 0.15 / 0.4 を入れると、奥行きは 0.050000 / 0.150000 / 0.400000。Output Back を切ると、面が 966 から 964 に減る。減るのはちょうど2枚で、これが裏側のふただった。点の数は変わらない。

角の丸め。polybevel の Distance (Offset) を上げてても、点 5,776・面 5,778 は動かない。丸めは形だけを変えるので、数では追えない。面積で測ると 1.689066 (0) → 1.656768 (0.005) → 1.643416 (0.01) → 1.629985 (0.02) → 1.617612 (0.04) と減っていき、0.08 以上は 1.616047 のまま6桁まで同じ。上げてても形がそれ以上変わらない。

丸めを入れると面が 966 から 5,778 が増える。Offset 0 でも同じ数になるので、増えるのは丸めの量ではなく、丸めノードを通したこと自体による。

平ら・厚み・角の丸め

文字「SP」、Font Size 1、LOD 1。3枚とも同じカメラ。

Houdini

font だけ。面は2枚で、厚みが無い。

Houdini

polyextrude の Distance 0.15。奥行きがちょうど 0.15 になる。

Houdini

polybevel の Distance 0.02。角に光の筋が出る。

A. font の Level of Detail

点の数だけが変わり、文字の大きさ（幅・高さ）は変わらない。

Level of Detail	点	面	幅	高さ	計算の時間
0.2	222	2	1.1220	0.7390	12.4 ms
0.5	518	2	1.1220	0.7390	0.7 ms
1 (既定)	962	2	1.1220	0.7390	0.7 ms
2	1,850	2	1.1220	0.7390	1.3 ms

Level of Detail	点	面	幅	高さ	計算の時間
4	3,626	2	1.1220	0.7390	3.4 ms

B. polyextrude の Distance と Output Back (LOD 1)

Output Back を切ると面が2枚減る。文字が2つなので、裏のふたが1文字につき1枚。

Distance	Output Back	点	面	奥行き
0.05	入り	1,924	966	0.050000
0.05	切り	1,924	964	0.050000
0.15	入り	1,924	966	0.150000
0.15	切り	1,924	964	0.150000
0.4	入り	1,924	966	0.400000
0.4	切り	1,924	964	0.400000

C. polybevel の Distance (厚み 0.15)

点と面の数は変わらないので、面ごとの面積を measure SOP で出して合計した。

Distance	点	面	面積の合計	計算の時間
0	5,776	5,778	1.689066	19.8 ms
0.005	5,776	5,778	1.656768	21.3 ms
0.01	5,776	5,778	1.643416	22.4 ms
0.02	5,776	5,778	1.629985	20.1 ms
0.04	5,776	5,778	1.617612	20.4 ms
0.08	5,776	5,778	1.616047	20.3 ms
0.15	5,776	5,778	1.616047	20.0 ms
0.3	5,776	5,778	1.616047	20.7 ms

わかったこと

- ・ font の Level of Detail は、点の数にほぼ比例する。0.2 / 0.5 / 1 / 2 / 4 で 222 / 518 / 962 / 1,850 / 3,626点。文字の大きさは変わらない（幅 1.1220・高さ 0.7390 が全部同じ）。曲線の刻みだけが変わる。

- ・ polyextrude の Distance は、そのまま奥行きになる。0.05 / 0.15 / 0.4 で、奥行きが 0.050000 / 0.150000 / 0.400000。

- ・ Output Back を切ると、裏のふたが消える。面が 966 → 964 と、ちょうど文字の数だけ減る。見た目は正面から変わらないが、裏から見ると穴が開いている。

- ・ 角の丸めは、点と面の数では追えない。polybevel の Distance を 0 から 0.3 まで変えても 5,776点・5,778面のまま。形だけが変わる。面積で測れば追える（1.689066 → 1.616047）。

- ・ 丸めは 0.08 で止まる。0.08 / 0.15 / 0.3 の面積が 1.616047 で6桁まで同じ。polybevel には「ぶつかったら止める」設定（Stop Loops / Detect Collisions）が既定で入っていて、文字の細い部分の幅までしか丸められない。エラーは出ない。

- ・丸めノードを通すだけで、面が 966 → 5,778 に増える。Distance 0 でも同じ数。丸めの量ではなく、通したこと自体で増える。
- ・計算はどれも軽い。font は LOD 4 でも 3.4ms、polyextrude は 3.4~4.2ms、polybevel は約 20ms。
- ・1つのプロセスで OpenGL の描画を3回続けたら、2枚目で「OpenGL Fatal Error」が出て止まった。1 プロセスにつき1枚だけ撮るようにしたら通った。レンダ를続けて撮るときは、プロセスを分ける。

次にやること

- ・ドミノを倒す（間隔をどこまで空けられるか）
- ・壁を崩す（破片の細かさと時間）
- ・文字を1文字ずつ分けて動かす

ならしても体積はほとんど減らない — 強さ160でも 99.60%、縮みは 0.40%だけ

2026-09-24

何を作ったか

smooth は点を近所の平均へ寄せる。凹凸が消えるのは狙い通りだが、同時に形が縮むと言われる。どれだけ縮むのかを測った。

球（60×60）に mountain をかけた形（体積 4.111425）に、Strength を 0 から 160 までかけた。体積は measure SOP の Volume を面ごとに足して出した。

既定の Strength 10 で体積は元の 99.9533%。16倍の 160 まで上げてても 99.6023% で、減ったのは 0.40% だけだった。「ならすと縮む」という言い方は、少なくともこの形とこのノードでは**体積の話としては当てはまらない**。

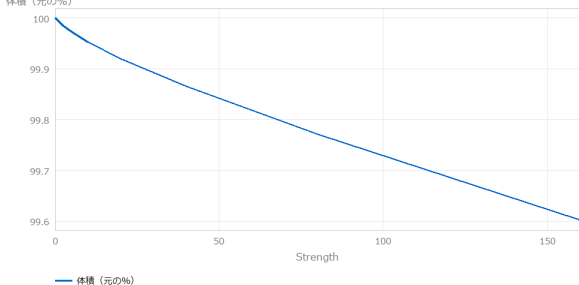
形そのものは動いている。元の面からのずれは強さ10で 0.002454、160で 0.006263。凹凸は確かに消えているが、出た所と入った所が相殺して、体積としては残る。

ならし方（Method）を変えても、縮みの差は小さい。強さ160で揃えると uniform が 99.6023%、scaledominant が 99.673%、curvaturDominant が 99.7338%。曲率を見るやり方がいちばん体積を保ち、同時にずれも小さかった。

Strength ごとの体積とずれ

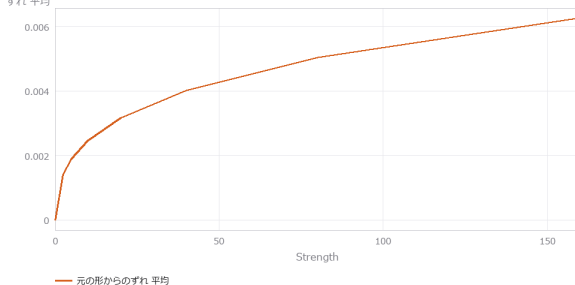
元は球（60×60）に mountain（高さ0.2）をかけた 3,540面、体積 4.111425。体積は measure の Volume を面ごとに足した値。ずれは各点から元の面までの最短距離。

ならしても、体積はほとんど減らない（強さ160で 99.60%）



強さを16倍にしても、体積は 0.40% しか減らない。

形は動いている。強さを上げるほどずれは増える



形は動いている。ずれは強さとともに増える。

Strength	体積	元の%	ずれ 平均	ずれ 最大	秒
0	4.111425	100.0%	0.000000	0.000000	0.000
2.5	4.110802	99.9849%	0.001414	0.012110	0.004
5	4.110319	99.9731%	0.001891	0.015560	0.003
10	4.109505	99.9533%	0.002454	0.019200	0.003
20	4.108155	99.9205%	0.003153	0.023063	0.004
40	4.105900	99.8656%	0.004013	0.027141	0.004
80	4.102029	99.7715%	0.005043	0.031824	0.004
160	4.095072	99.6023%	0.006263	0.037208	0.005

ならし方 (Method) を変えたとき (Strength 160 で揃える)

3つのやり方を、同じ強さで比べた。

Method	体積	元の%	ずれ 平均	ずれ 最大
uniform	4.095072	99.6023%	0.006263	0.037208
scaledominant	4.097979	99.673%	0.005568	0.035997
curvatedominant	4.100479	99.7338%	0.005101	0.034963

わかったこと

- ・縮みは 0.40% だけ。Strength 160 で体積は元の 99.6023%。既定の 10 では 99.9533%。「ならすと縮む」は、体積の話としてはこの形では出なかった。
- ・形は動いている。ずれの平均は強さ10で 0.002454、160で 0.006263。凹凸は消えるが、出た所と入った所が相殺している。
- ・曲率を見るやり方がいちばん保つ。強さ160で、curvatedominant が 99.7338%、uniform が 99.6023%。ずれも 0.005101 と 0.006263 で、曲率を見るほうが小さい。

次にやること

- ・薄い形（板や布）では縮みが出るのか
- ・強さをもっと上げると、どこで体積が落ち始めるのか
- ・vdbsmooth との縮み方の違い

隙間 0.01 の継ぎ目は、Snap Distance が 0.01 になって初めて閉じる — 0.0095 では閉じない

2026-09-24

何を作ったか

fuse は Snap Distance より近い点をひとつにまとめる。では「より近い」の境目はどこなのか。そして距離を上げていくと、狙った継ぎ目だけが閉じるのか、細かい所まで潰れてしまうのか。2つの場面で測った。

ひとつ目は、板を2枚ちょうど 0.01 だけ離して並べた形（242点）。Snap Distance を 0 から上げていくと、0.0095 までは1点も合わさらず、0.01 でいきなり 11点（継ぎ目の1列ぶん）が合わさった。 **距離が隙間と同じ値になったところで閉じる。 ** それより手前では、どれだけ近くても閉じない。

ふたつ目は、球（50×50、2402点）にそのまま fuse をかけた場合。距離 0.01 で 50点、0.02 で 118点が減るが、 **形は動かない**（ずれ 0.0）。極のあたりに同じ位置の点が重なっていて、それが掃除されている。

形が動き出すのは 0.04 を超えてから。0.08 では点が元の 41.299% まで減り、ずれは 0.000028。0.16 では 15.279% まで減り、ずれは 0.002116 で、形そのものが崩れる。

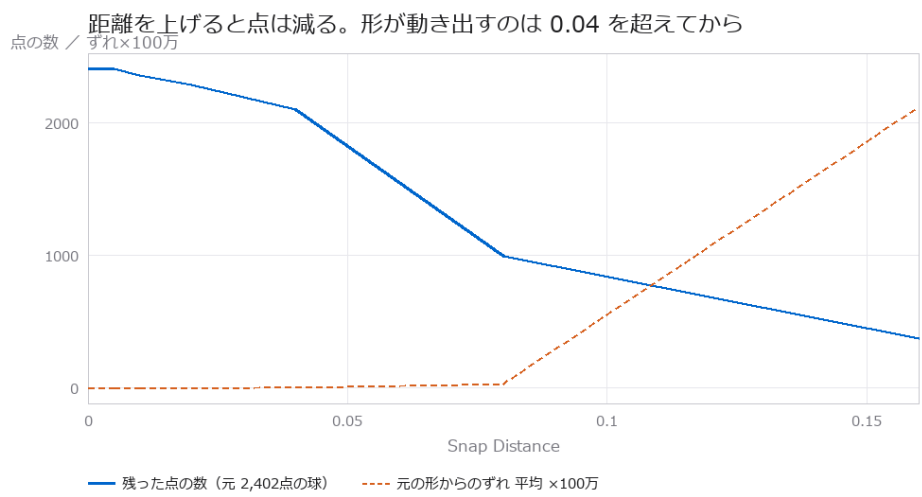
板2枚（隙間 0.01）の継ぎ目が閉じる境目

1辺1.0の板を11×11で2枚、x方向に 0.01 離して並べた。継ぎ目に向き合う点は左右で11個ずつ。

Snap Distance	残った点	合わさった点
0	242	0
0.001	242	0
0.005	242	0
0.009	242	0
0.0095	242	0
0.01	231	11
0.011	231	11
0.02	231	11
0.05	231	11

球にそのままかけたとき

球（50×50）の 2,402点。ずれは各点から元の面までの最短距離の平均。距離 0.01・0.02 では点が減ってもずれが 0 のままで、重なっていた点だけが掃除されたことが分かる。



点は早い段階から減るが、形が動き出すのは 0.04 を超えてから。

Snap Distance	点	元の%	面	ずれ 平均
0	2402	100.0%	2450	0.000000
0.005	2402	100.0%	2450	0.000000
0.01	2352	97.918%	2400	0.000000
0.02	2284	95.087%	2348	0.000000
0.04	2100	87.427%	2164	0.000002
0.08	992	41.299%	1942	0.000028
0.16	367	15.279%	657	0.002116

わかったこと

- ・ 境目は隙間と同じ値。隙間 0.01 の継ぎ目は、Snap Distance 0.0095 では閉じず、0.01 で閉じた。「より近い」は、距離がちょうど同じときも含む。
- ・ 点が減っても形が変わらないことがある。球では距離 0.01 で 50点、0.02 で 118点が減るが、ずれは 0.0 のまま。同じ位置に重なっていた点が掃除されただけ。
- ・ 形が崩れ始めるのは 0.04 より上。0.04 でずれ 0.000002、0.08 で 0.000028、0.16 で 0.002116。掃除のつもりで大きな値を入れると、形まで持っていかれる。

次にやること

- ・ 隙間が斜めのときも、境目は同じ値になるか
- ・ Snap To (どこへ寄せるか) を変えると位置がどうずれるか

boolean は答えと6桁まで一致する ー 差の正体は円ではなく多角形だった

2026-09-24

何を作ったか

1辺 1.0 の箱に、半径 0.25 の円柱の穴を通す。答えは計算で出せる。円で考えれば 0.80365。ただし Houdini の円柱は多角形なので、80角形で考えた答えは 0.803852 になる。

この「答えの分かっている形」を3通りで作って、体積を突き合わせた。

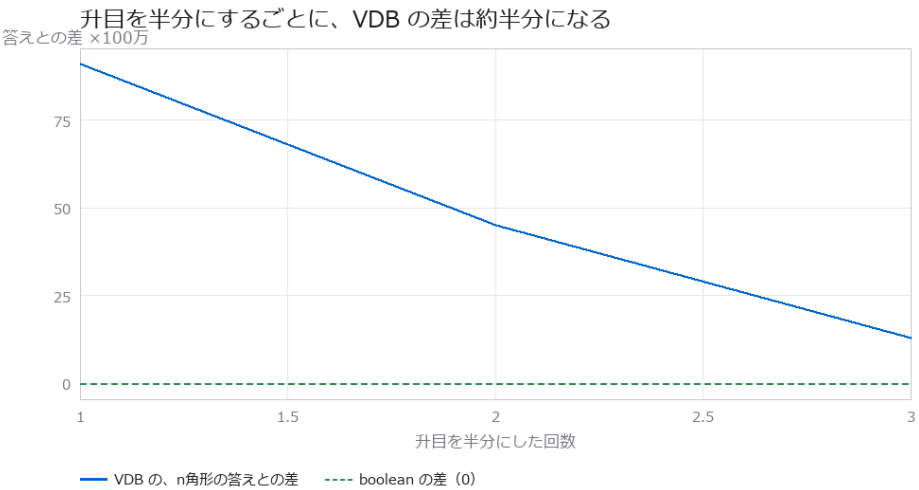
boolean と cookie は、どちらも 0.803852。 **80角形の答えとの差は +0.000000**、つまり6桁まで一致する。円の答えとの差 +0.000202 は、アルゴリズムのずれではなく多角形近似そのものだった。

念のため、円柱の辺の数を 12 / 40 / 80 / 200 と変えて確かめた。4通りすべてで、出た体積は n角形の答えと差 0.000000 で一致した。辺を増やすほど円の答えに近づく（12角形で +0.008850、200角形で +0.000032）。

VDB は升目の細かさで決まる。升目 0.02 で +0.000091、半分にするとに差も約半分になり、0.005 で +0.000013。ただし面の数は 307,968 で、boolean の 86 面に対して桁が違う。 **平らな面で足りる形に VDB を使うと、精度で負けて重さで負ける。 **

3つの道筋の結果（円柱は80角形）

箱は poly の 6面、円柱は poly の 82面。「n角形との差」は 80角形で考えた答え 0.803852 との差、「円との差」は円で考えた答え 0.80365 との差。

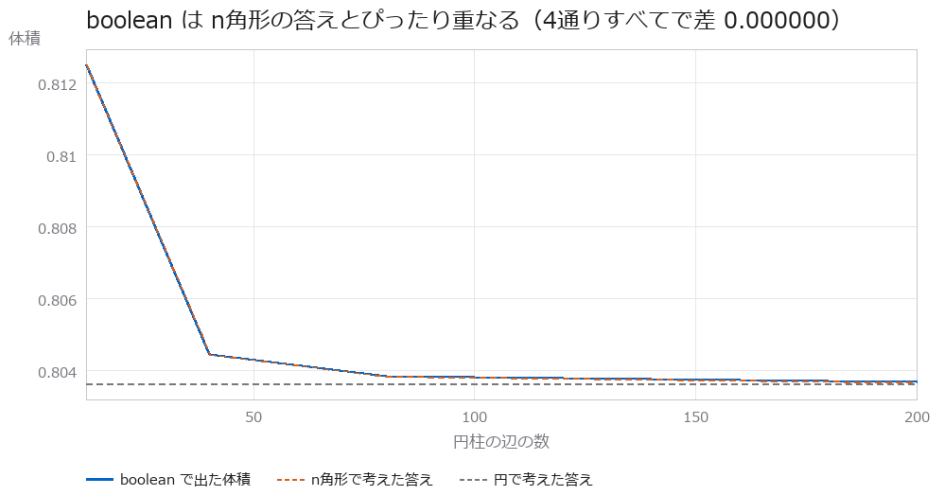


VDB の差は升目を半分にするとに約半分。boolean は最初から差が無い。

道筋	細かさ	面	秒	体積	n角形との差	円との差
boolean	Subtract	86	0.006	0.803852	+0.000000	+0.000202
cookie	A minus B	86	0.001	0.803852	+0.000000	+0.000202
vdb	升目 0.02	19,728	0.041	0.803944	+0.000091	+0.000293
vdb	升目 0.01	77,920	0.051	0.803897	+0.000045	+0.000246
vdb	升目 0.005	307,968	0.106	0.803866	+0.000013	+0.000215

円柱の辺の数を変える (boolean)

辺の数 n を変えると、答えも n 角形の式で変わる。出た値がその式を追うかを見た。



4通りすべてで、答えの線と完全に重なっている。

辺の数	面	体積	n角形の答え	差	円との差
12	18	0.812500	0.812500	+0.000000	+0.008850
40	46	0.804457	0.804457	+0.000000	+0.000806
80	86	0.803852	0.803852	+0.000000	+0.000202
200	206	0.803683	0.803683	+0.000000	+0.000032

わかったこと

- boolean と cookie は答えと6桁まで一致する。どちらも 0.803852、80角形の答えとの差は +0.000000。cookie は 0.0011秒、boolean は 0.0063秒だった。
- 円との差は多角形近似のぶん。12角形で +0.008850、200角形で +0.000032。アルゴリズムを疑う前に、円柱の辺の数を見るほうが早い。
- VDB は精度でも重さでも負ける（この形では）。升目 0.005 で差 +0.000013、面は 307,968。boolean は 86面で差 +0.000000。平らな面で表せる形なら、VDB を通す理由は無い。
- つまずいた点。箱と円柱の Primitive Type を poly 以外 (mesh / polymesh) にすると、boolean も cookie も黙って何もしない。エラーも警告も出ず、箱がそのまま出てくる。最初これに気づかず、体積 1.000000 のまま「差 +24%」という表を作ってしまった。答えの分かっている形で試していたから気づけた。

次にやること

- 曲面（球どうし）の引き算では、どの道筋がいちばん答えに近いのか
- 面が重なっている・接しているときに boolean が崩れる条件
- VDB の差が升目の1乗で減るのか2乗で減るのか（もっと細かくして確かめる）

立方体を割り続けると、体積は約32.76%に落ち着く — bilinear だけはちょうど1.000000のまま

2026-09-24

何を作ったか

subdivide (Catmull-Clark) は、割るたびに形を滑らかにする。立方体を割り続けると、どこまで縮んで、どこで止まるのか。1辺 1.0 の立方体 (体積 1.000000) を深さ 0 から 6 まで割って測った。

深さ1で 42.7083%、深さ2で 35.0192%、深さ6で 32.7638%。1段ごとの減りは 0.572917 / 0.076892 / 0.017111 / 0.004155 / 0.001031 / 0.000257

で、**前の段の約4分の1ずつ**小さくなる (比は 7.45 / 4.49 / 4.12 / 4.03 / 4.01) 。

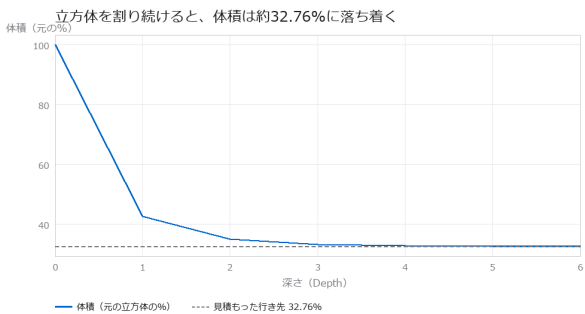
この比が 1/4 のまま続くとして残りを足すと、行き先は約 0.327552 (元の 32.76%)。ここは測った値ではなく見積もり。深さ6の実測は 0.327638 で、見積もりとの差は +0.000086。

測り方が正しいことは、割り方を変えると確かめられる。**osdbilinear は深さ3でも体積 1.0** で、元のまま1ミリも変わらない。滑らかにせず、ただ割るだけの方式なので体積が変わらないのが正しい。ここが 1.000000 で出たので、他の数字も信じられる。

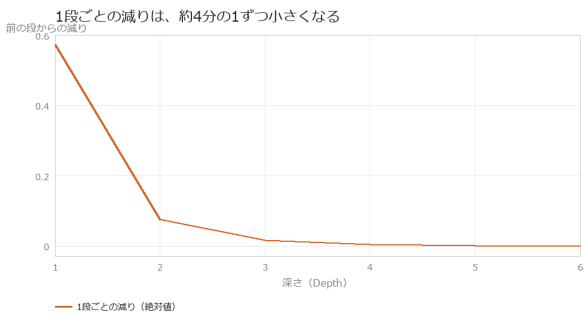
houdini と osdcc は同じ値 (どちらも 0.333081)。mantra は少し小さく 0.322114。osdloop は三角形に割るので面の数が倍 (768面) で、体積は 0.376523 と大きめに残る。

深さごとの面の数と体積 (osdcc)

元は poly の立方体 (6面・体積 1.000000)。体積は measure の Volume を面ごとに足した値。「前の段から」は、ひとつ浅い深さとの差。



深さ3でほぼ落ち着き、そこからはほとんど動かない。



1段ごとの減りは約4分の1ずつ。等比で残りを足すと行き先が出る。

深さ	面	点	秒	体積	元の%	前の段から
0	6	8	0.000	1.000000	100.0%	+0.000000
1	24	26	0.000	0.427083	42.7083%	-0.572917
2	96	98	0.000	0.350192	35.0192%	-0.076892
3	384	386	0.000	0.333081	33.3081%	-0.017111
4	1,536	1,538	0.001	0.328926	32.8926%	-0.004155
5	6,144	6,146	0.002	0.327895	32.7895%	-0.001031
6	24,576	24,578	0.008	0.327638	32.7638%	-0.000257

割り方 (Algorithm) を5通り (深さ3で揃える)

osdbilinear は滑らかにせず割るだけなので、体積が変わらないのが正しい。測り方の裏付けになる。

Algorithm	面	点	秒	体積	元の%
houdini	384	386	0.001	0.333081	33.3081%
mantra	384	386	0.001	0.322114	32.2114%
osdcc	384	386	0.000	0.333081	33.3081%
osdloop	768	386	0.000	0.376523	37.6523%
osdbilinear	384	386	0.000	1.000000	100.0%

わかったこと

- ・行き先は元の約 32.76%。深さ6の実測は 32.7638%。1段ごとの減りが約1/4ずつなので、残りを等比で足して見積もった（この行き先の値は実測ではない）。

- ・深さ3でほぼ決まる。深さ3で 33.3081%、深さ6で 32.7638%。差は 0.54ポイントしかない。面の数は 384 から 24,576
へ64倍になる。深さを上げて形はほとんど変わらないのに、重さだけが増える。

- ・osdbilinear はちょうど 1.000000
だった。滑らかにせず割るだけなので、体積が変わらないのが正しい。測り方（measure の Volume を面ごとに足す）が正しいことの裏付けになる。

- ・houdini と osdcc は同じ値。どちらも 0.333081。mantra は 0.322114 で少し小さく、osdloop は三角形に割るため面が倍（768面）で 0.376523 と大きめに残る。

次にやること

- ・Catmull-Clark の極限体積は、計算で求められる値と一致するか
- ・球や円柱では、深さいくつで落ち着くか
- ・Crease Weight を入れると、どこまで縮みを止められるか

peak は式どおりに動く — 球は $((r+d)/r)^3$ 、立方体は $(1+2d/\sqrt{3})^3$ に6桁一致

2026-09-24

何を作ったか

peak は点を、その点の法線の向きへ Distance だけ動かす。球なら半径が r から $r+d$ になるだけなので、体積は $((r+d)/r)^3$ 倍になるはず。球は多角形なので体積そのものは真の球より小さいが、**倍率なら分割の粗さに関係なく式に合う**はず。

3つの粗さ (20×20 / 60×60 / 120×120) × 6つの距離 = 18通りを測った。差はすべて 0.000000 台で、いちばん大きい差でも 0.000003。20×20 の球は真の球の 97.692% しかない粗さだが、倍率は $d=1.0$ で 7.999997 (式は 8.0) 。**粗さは倍率に影響しない**。

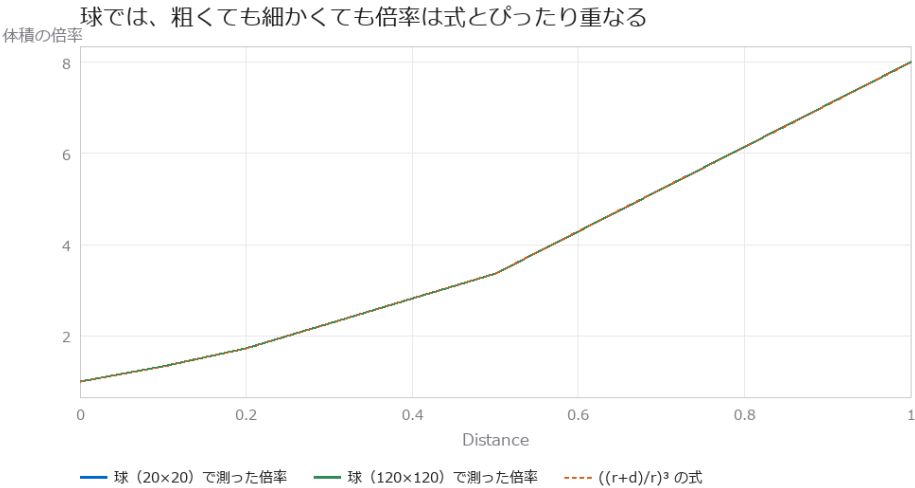
立方体では話が変わる。立方体の点の法線は、隣り合う3面の平均で $(1,1,1)/\sqrt{3}$ を向く。だから角は各軸に $d/\sqrt{3}$ しか動かず、1辺は $1+2d$ ではなく $**1 + 2d/\sqrt{3}**$ になる。

$d=1.0$ のとき、1辺が $1+2d$ になったつもりだと 27倍を期待するが、実測は 10.003702。 $1 + 2d/\sqrt{3}$ の式では 10.003702 で、差は -0.000001。 **思い違いのほう が 16.99 もずれていた**。

peak が「法線の向きに動かす」と書かれているとき、その法線は面の向きではなく点の向き。角のある形では、そこが効いてくる。

球：測った倍率と $((r+d)/r)^3$

半径 1.0 の球に normal を通してから peak をかけた。真の球の体積は 4.18879。倍率は、同じ粗さの $d=0$ の体積で割った値。



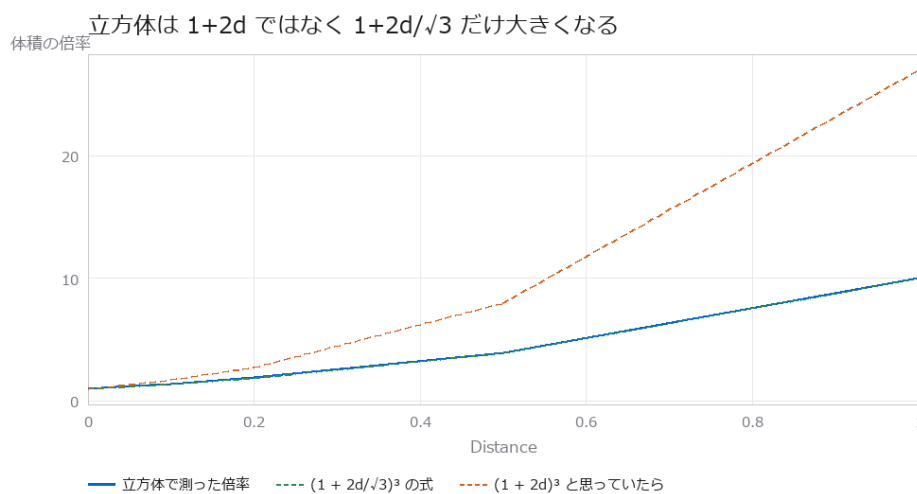
3つの粗さの線が、式の線と完全に重なっている。

球の粗さ	Distance	体積	倍率	式	差
20×20	0	4.092129	1.000000	1.000000	+0.000000
20×20	0.05	4.737151	1.157625	1.157625	-0.000000
20×20	0.1	5.446624	1.331000	1.331000	-0.000000
20×20	0.2	7.071198	1.728000	1.728000	-0.000000
20×20	0.5	13.810933	3.374999	3.375000	-0.000001

球の粗さ	Distance	体積	倍率	式	差
20×20	1	32.737022	7.999997	8.000000	-0.000003
60×60	0	4.178176	1.000000	1.000000	+0.000000
60×60	0.05	4.836760	1.157625	1.157625	+0.000000
60×60	0.1	5.561152	1.331000	1.331000	-0.000000
60×60	0.2	7.219887	1.728000	1.728000	+0.000000
60×60	0.5	14.101343	3.375000	3.375000	+0.000000
60×60	1	33.425405	8.000000	8.000000	+0.000000
120×120	0	4.186147	1.000000	1.000000	+0.000000
120×120	0.05	4.845988	1.157625	1.157625	-0.000000
120×120	0.1	5.571762	1.331000	1.331000	+0.000000
120×120	0.2	7.233662	1.728000	1.728000	+0.000000
120×120	0.5	14.128246	3.375000	3.375000	+0.000000
120×120	1	33.489176	8.000000	8.000000	+0.000000

立方体：1+2d ではなく $1+2d/\sqrt{3}$

立方体の点の法線は3面の平均で $(1,1,1)/\sqrt{3}$ を向く。角は各軸に $d/\sqrt{3}$ だけ動く。



$1+2d$ だと思っていると大きく外す。実測は $1+2d/\sqrt{3}$ の線に重なる。

Distance	1辺	体積	$1+2d/\sqrt{3}$ の式	差	$1+2d$ の式	その差
0	1.000000	1.000000	1.000000	+0.000000	1.000000	+0.000000
0.05	1.057735	1.183397	1.183398	-0.000000	1.331000	-0.147603
0.1	1.115470	1.387950	1.387950	-0.000000	1.728000	-0.340050
0.2	1.230940	1.865137	1.865137	-0.000000	2.744000	-0.878863
0.5	1.577350	3.924501	3.924501	-0.000000	8.000000	-4.075499
1	2.154701	10.003702	10.003702	-0.000001	27.000000	-16.996298

わかったこと

- ・球では18通りすべて式に一致した。いちばん大きい差でも 0.000003。20×20 の粗い球（真の球の 97.692%）でも、倍率は細かい球とまったく同じだった。

- ・倍率で見ると、分割の粗さは消える。体積そのものは粗さで変わるが、同じ形の $d=0$ で割れば粗さの分が打ち消える。「答えと突き合わせる」ときに使える手。

- ・立方体は $1+2d/\sqrt{3}$ 。 $d=1.0$ で実測 10.003702、式 10.003702（差 -0.000001）。 $1+2d$ だと思っていると 27倍を期待して 17.00 も外す。

- ・法線は面の向きではなく点の向き。立方体の角の点は3面の平均を向くので、斜めに出る。角のある形を peak で厚くすると、面は動かず角だけが出っばる。

次にやること

- ・ polyextrude で厚みを付けたときは、どの式に合うか
- ・ 法線を面ごとに持たせた（頂点法線）状態で peak をかけるとどうなるか
- ・ Mask で効きを弱めたとき、倍率はどう崩れるか

面積の測りは、平らな面ならぴったりに合う ー 球の足りない分は分割の2乗で減る

2026-09-24

何を作ったか

面積は「測るための道具」としてよく使う。散らす濃さを決めたり、重さを面積から出したり。その道具そのものの精度を、答えの分かっている形で確かめた。

1辺 2.0 の平らな板は、面積 4.0 がぴったり出る。分割を 2 から 201 まで変えても差は最大 0.000000032（小数の丸めの範囲）。**分割の数は面積に影響しない。**

球は違う。半径 1.0 の球の答えは 12.566371 だが、多角形なので必ず小さく出る。分割 8 では 11.635271（7.40945% 足りない）、分割 240 で 12.565382（0.00787% 足りない）。

足りない分の減り方は 4.075 / 3.558 / 4.032 / 4.017 / 4.009 で、**分割を倍にすると4分の1**になる。誤差が分割の2乗で減るということで、細かくする効きは良い。

実用の目安として、分割 60（3,540面）で足りない分は 0.12675%。面積を使って何かを決めるぶんには十分だが、答えとして出すなら、多角形のぶん小さく出ていることを覚えておく。

平らな板（1辺 2.0、答えは 4.000000）

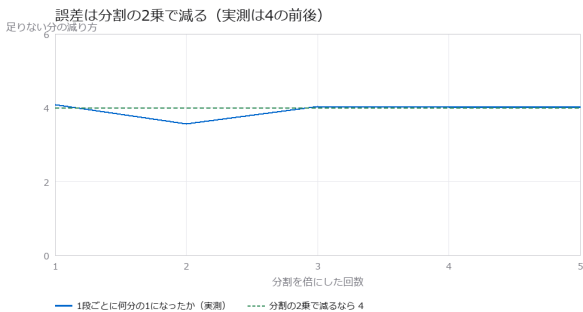
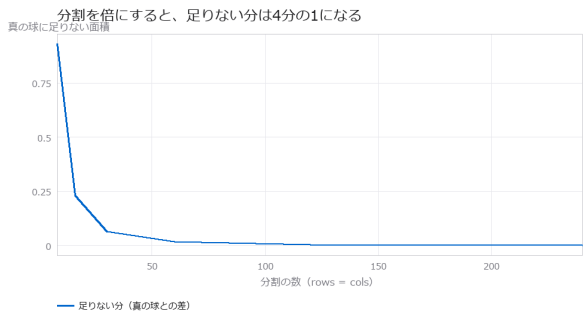
grid の分割を変えた。差は小数の丸めの範囲に収まる。

分割	面	面積	答えとの差	秒
2	1	4.000000	+0.000000000	0.000
5	16	4.000000	+0.000000000	0.000
11	100	4.000000	-0.000000015	0.001
51	2,500	4.000000	+0.000000032	0.026
201	40,000	4.000000	-0.000000031	0.403

球（半径 1.0、答えは 12.566371）

polymesh の球。rows と cols

を同じ数にした。「何分の1」は、前の段の足りない分をこの段の足りない分で割った値。



分割を倍にすると、足りない分は4分の1になる。

減り方は4の前後。誤差は分割の2乗で減っている。

分割	面	面積	足りない分	%	何分の1	秒
8	56	11.635271	0.931099	7.40945%	—	0.001

分割	面	面積	足りない分	%	何分の1	秒
16	240	12.337864	0.228507	1.8184%	4.075	0.003
30	870	12.502151	0.064219	0.51104%	3.558	0.009
60	3,540	12.550442	0.015928	0.12675%	4.032	0.038
120	14,280	12.562405	0.003965	0.03155%	4.017	0.145
240	57,360	12.565382	0.000989	0.00787%	4.009	0.602

わかったこと

- ・平らな面はぴったり。1辺2.0の板は、分割 2 から 201 までどれも 4.000000。差は最大 0.000000032 で、小数の丸めの範囲。
- ・球は必ず小さく出る。多角形が球の内側を通るので、分割 8 で 7.40945%、分割 60 で 0.12675%、分割 240 で 0.00787% 足りない。
- ・足りない分は分割の2乗で減る。実測 4.075 / 3.558 / 4.032 / 4.017 / 4.009。分割を倍にすると4分の1になるので、細かくする効きは良い。

次にやること

- ・曲率の測り（Curvature）も同じように収束するか
- ・体積の測りは、面積と同じ減り方をするか

押し出した体積は式どおり — 面積×距離、Insetを入れても四角錐台の式に6桁一致

2026-09-24

何を作ったか

polyextrude で平らな板を厚くすると、できる立体の体積は「元の面積 × 押し出した距離」になるはず。答えの分かっている形で突き合わせた。

1辺 2.0 の板（面積 4.0）を 0.1 から 2.0 まで押し出した。5通りすべてで差は 0.000000。 **式どおり。 **

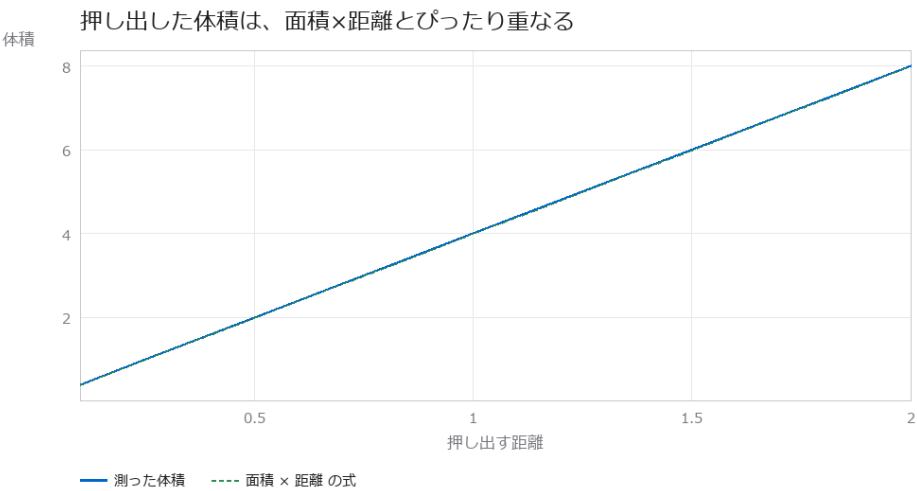
Inset を入れると上の面が小さくなり、立体は四角錐台になる。1辺 L の正方形を i だけ内側に寄せると、上の1辺は L-2i。四角錐台の体積は $d/3 \times (L^2 + (L-2i)^2 + L(L-2i))$ になる。

Inset を 0 から 0.9 まで（上の1辺 2.0 から 0.2 まで）変えて測ったところ、5通りすべてで差は 0.000000。 **こちらも式どおり。 **

Inset 0.9 では体積が 1.48 で、Inset なしの 4.0 に対して 37.0%。上の面をほとんど潰しても、体積は3分の1までしか減らない（錐に近づくので、底のぶんが残る）。

押し出す距離を変える（1辺 2.0、面積 4.0）

grid（1面）に polyextrude をかけ、前・後・側面のすべてを出した。体積は measure の Volume を面ごとに足した値の絶対値。

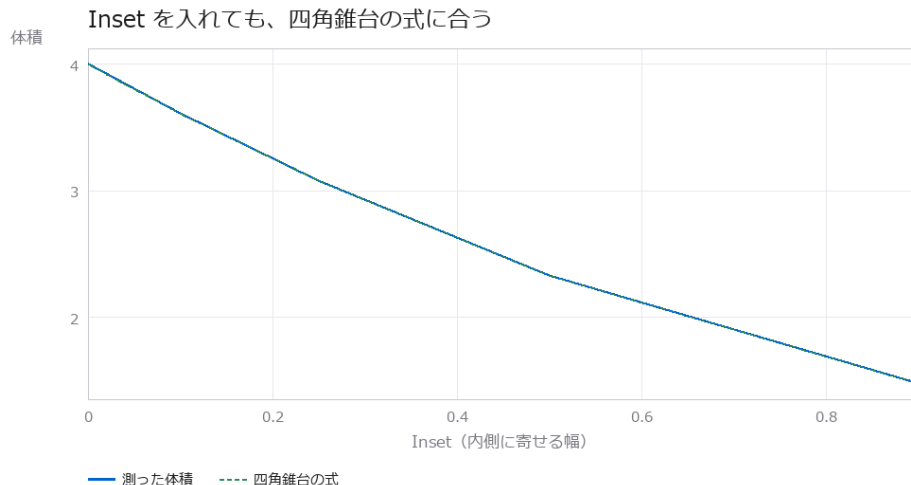


2本の線が完全に重なっている。

距離	面	体積	面積×距離	差	秒
0.1	6	0.400000	0.400000	+0.000000	0.001
0.25	6	1.000000	1.000000	+0.000000	0.000
0.5	6	2.000000	2.000000	+0.000000	0.000
1	6	4.000000	4.000000	+0.000000	0.000
2	6	8.000000	8.000000	+0.000000	0.000

Inset を変える（距離は 1.0 で揃える）

上の1辺は $2.0 - 2 \times \text{Inset}$ 。四角錐台の式は $d/3 \times (L^2 + \text{上}^2 + L \times \text{上})$ 。



Inset を入れても、四角錐台の式の線に重なる。

Inset	上の1辺	体積	四角錐台の式	差	Insetなしとの差
0	2.00	4.000000	4.000000	+0.000000	+0.000000
0.1	1.80	3.613333	3.613333	-0.000000	-0.386667
0.25	1.50	3.083333	3.083333	-0.000000	-0.916667
0.5	1.00	2.333333	2.333333	+0.000000	-1.666667
0.9	0.20	1.480000	1.480000	+0.000000	-2.520000

わかったこと

- ・面積×距離で合う。5通りすべてで差は 0.000000。押し出した体積を式で見積もれるので、重さや量の計算にそのまま使える。
- ・Inset も四角錐台の式で合う。5通りすべてで差は 0.000000。上の面が小さくなるぶんは、式のとおり減る。
- ・上の面を潰しても、体積は3分の1までしか減らない。Inset 0.9（上の1辺 0.2）で 1.48、Inset なしの 37.0%。錐に近づくので、底の面のぶんが残る。
- ・実験100と合わせて。peak は法線の向きに動かすので、角のある形では式が変わる（立方体は $1+2d/\sqrt{3}$ ）。polyextrude は面の向きに押し出すので、面積×距離で素直に効く。厚みを付けたいだけなら polyextrude のほうが読みやすい。

次にやること

- ・曲面（球の一部）を押し出したときは、どの式に合うか
- ・Divisions を入れて段を作ったとき、体積は変わらないか

Rotate Order の xyz は「x から順に掛ける」 — 6通りで最大 0.725534 (約42.5°) ずれる

2026-09-24

何を作ったか

Transform には Rotate Order (xyz / xzy / yxz / yzx / zxy / zyx) がある。同じ角度を入れても、掛ける順番で行き先が変わる。どれだけ変わるのかと、Houdini の並びが数式でいう何の順番なのかを確かめた。

点を1つ (1.0, 0.0, 0.0) に置き、角度 (30.0, 40.0, 50.0) (度) で回した。6通りすべての行き先を測り、自分で行列を掛けて出した答えと突き合わせた。

**xyz は「x を先に掛け、次に y、最後に z」だった。 **6通りすべてで、この掛け順との差は 0.000000。逆から掛けた式では合わない (差は 0.16~0.73) 。行列で書けば xyz は $R_z \cdot R_y \cdot R_x$ になる。

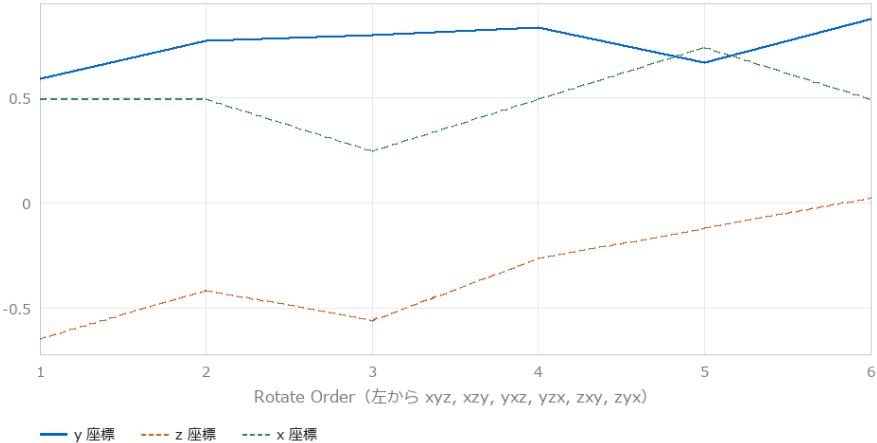
ばらつきは小さくない。もっとも離れた2つは **xyz と zyx** で 0.725534。点は原点から 1.0 の距離にあるので、**約 42.5° ぶん開いている**。もっとも近い2つ (xzy と yzx) でも 0.162832 離れる。

1軸だけ回すときは順番に関係ない。y だけ 40° 回すと、6通りの差は最大 0.000000000 だった。「順番を気にしなくてよい」のは、角度が1つだけ入っているときに限る。

6通りの行き先 (点 (1.0, 0.0, 0.0) を (30.0, 40.0, 50.0)° 回す)

「左から掛けた式との差」は、並びの順に $R_x \rightarrow R_y \rightarrow R_z$ と掛けた (行列では $R_z \cdot R_y \cdot R_x$) 答えとの距離。「右から」は逆順に掛けた答えとの距離。

同じ角度でも、順番が違えば行き先が違う (xyz / xzy / yxz / yzx / zxy / zyxの順)
回したあとの座標



x 座標は3通りの値しか取らないが、y と z は6通りすべてで違う。

Rotate Order	x	y	z	左から掛けた式との差	右から掛けた式との差
xyz	+0.492404	+0.586824	-0.642788	0.000000	0.725534
xzy	+0.492404	+0.766044	-0.413176	0.000000	0.162832
yxz	+0.246202	+0.793412	-0.556670	0.000000	0.671006
yzx	+0.492404	+0.829598	-0.263258	0.000000	0.162832

Rotate Order	x	y	z	左から掛けた式との差	右から掛けた式との差
zxy	+0.738606	+0.663414	-0.119764	0.000000	0.671006
zyx	+0.492404	+0.870002	+0.025201	0.000000	0.725534

6通りのあいだの距離（離れている順）

点は原点から 1.0 の距離にある。距離 1.0 なら 60° ぶん開いていることになる。

組	距離	開いた角度
xyz $\leftrightarrow$ zyx	0.725534	42.5°
yxz $\leftrightarrow$ zxy	0.671006	39.2°
yxz $\leftrightarrow$ zyx	0.636440	37.1°
xyz $\leftrightarrow$ zxy	0.583125	33.9°
xyz $\leftrightarrow$ yzx	0.450535	26.0°
xzy $\leftrightarrow$ zyx	0.450535	26.0°
xzy $\leftrightarrow$ zxy	0.396534	22.9°
yxz $\leftrightarrow$ yzx	0.384728	22.2°
zxy $\leftrightarrow$ zyx	0.352575	20.3°
xyz $\leftrightarrow$ yxz	0.332731	19.2°
yzx $\leftrightarrow$ zxy	0.329884	19.0°
xyz $\leftrightarrow$ xzy	0.291276	16.7°
yzx $\leftrightarrow$ zyx	0.291276	16.7°
xzy $\leftrightarrow$ yxz	0.286278	16.5°
xzy $\leftrightarrow$ yzx	0.162832	9.3°

わかったこと

- ・xyz は「x から順に掛ける」。6通りすべてで、この掛け順との差は 0.000000。行列では $R_z \cdot R_y \cdot R_x$ 。逆順の式では合わない。
- ・順番の違いは約 42.5° に達する。xyz と zyx で 0.725534（原点から 1.0 の点で）。同じ数字を入れたのに、これだけ別の場所へ行く。
- ・1軸だけなら順番は関係ない。y だけ 40° 回すと、6通りの差は最大 0.000000000。2軸以上に角度が入っているときだけ、順番が効く。
- ・x 座標は3通りしか取らない。x を最初に掛ける並び（xyz / xzy）と、x を最後に掛ける並び（yzx / zyx）で値が揃う。回す軸と座標の関係で、一部の成分だけが一致することがある。

次にやること

- ・Transform Order (srt / str / rst …) を変えたときのずれ
- ・クォータニオンで同じ回転を作ると、順番の問題は消えるのか
- ・回転を混ぜる（補間する）とき、順番の違いがどう出るか

resample の点の数は切り上げ — 「最大の長さ」は本当に最大だった

2026-09-24

何を作ったか

resample の Maximum Segment Length に s を入れると、長さ 7.0 の線は何点になるのか。切り上げなら $\text{ceil}(L/s)+1$ 、四捨五入なら $\text{round}(L/s)+1$ 。割り切れない値を混ぜて9通り測った。

結果は **9通りすべて切り上げ** 。2つの式が分かれるのは 2 か所（刻み 0.3 と 3）で、そこでも切り上げのほうが合った。

できた刻みは、指定した値より必ず短いか等しい。たとえば刻み 3.0 を指定すると4点になり、実際の刻みは 2.333333。刻み 1.5 では 1.4。 ** 「最大の長さ」という名前のとおりで、指定を超える区間はできない。 ** 線の全長はどの刻みでも 7.0 のまま変わらなかった。

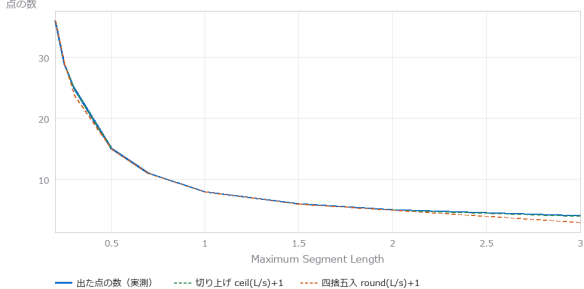
carve も測った。0～1 の範囲で線の一部を取り出す箱で、長さは「元の長さ × (終わり - 始まり)」になるはず。5通りすべてで差は 0.000000。0.5～0.5（幅ゼロ）を指定すると、長さ 0・点 0 で何も出てこない。

つまずいた点: carve の First U / Second U というつまみは**「使うかどうか」の入切**で、値は domainu1 / domainu2 という別のつまみに入っている。既定では First U だけが入で domainu1 が 0.25 なので、何も触らないと 0.25～1.0 が取り出される（長さ 5.25）。画面のラベルが同じ名前で2つ並ぶので、スクリプトから触るときに間違えやすい。

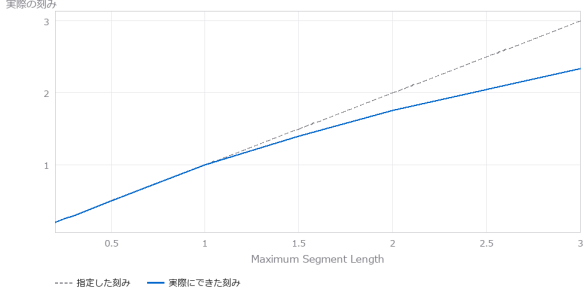
resample の刻みと点の数（長さ 7.0 の線）

元は line（2点）。長さは measure の Perimeter を面ごとに足した値。「実際の刻み」は $\text{長さ} \div (\text{点の数} - 1)$ 。

点の数は切り上げ。四捨五入の線とは2か所で分かれる



実際の刻みは、指定より必ず短いか等しい (Maximum は本当に最大)



実測は切り上げの線に重なる。四捨五入の線とは2か所で分かれる。

実際の刻みは、指定した値を超えない。

刻み	L/s	点	切り上げ	四捨五入	実際の刻み	長さ
0.2	35.0000	36	36	36	0.200000	7.000000
0.25	28.0000	29	29	29	0.250000	7.000000
0.3	23.3333	25	25	24	0.291667	7.000000
0.5	14.0000	15	15	15	0.500000	7.000000
0.7	10.0000	11	11	11	0.700000	7.000000

刻み	L/s	点	切り上げ	四捨五入	実際の刻み	長さ
1	7.0000	8	8	8	1.000000	7.000000
1.5	4.6667	6	6	6	1.400000	7.000000
2	3.5000	5	5	5	1.750000	7.000000
3	2.3333	4	4	3	2.333333	7.000000

carve で取り出す範囲と長さ（元は 7.0）

値は domainu1 / domainu2 に入れる（First U / Second U は入切）。

始まり	終わり	点	長さ	式	差
0	1	2	7.000000	7.000000	+0.000000
0	0.5	2	3.500000	3.500000	+0.000000
0.25	0.75	2	3.500000	3.500000	+0.000000
0.1	0.9	2	5.600000	5.600000	-0.000000
0.5	0.5	0	0.000000	0.000000	+0.000000

わかったこと

- ・点の数は切り上げ。9通りすべて $\text{ceil}(L/s)+1$ と一致した（例外なし）。四捨五入と分かれる所でも切り上げだった。
- ・「最大の長さ」は本当に最大。刻み 3.0 を指定すると実際は 2.333333、1.5 なら 1.4。指定を超える区間はできない。細かく刻みたいなら、その数字より短くなることを前提にする。
- ・carve は式どおり。5通りすべてで差は 0.000000。幅ゼロを指定すると点も出てこない（長さ 0・点 0）。
- ・つまずいた点。carve の First U / Second U は「使うかどうか」の入切で、値は domainu1 / domainu2 にある。既定は First U だけ入・domainu1 が 0.25 なので、何も触らないと 0.25～1.0（長さ 5.25）になる。画面に同じラベルが2つ並ぶ型のつまみは、スクリプトから触るとき要注意。

次にやること

- ・曲がった線でも、切り上げの規則は同じか
- ・Maximum Segment Length ではなく点の数を指定したときの刻み方
- ・carve を時間で動かして線を伸ばすとき、点の数はどう変わるか

spiral の長さは折れ線の式に6桁一致 — 1巻き50分割で 0.06% 短い

2026-09-24

何を作ったか

spiral を多角形で出して、長さを式と突き合わせた。半径を一定にすると、1巻きを N 分割した折れ線の1区間は、横に $2r \cdot \sin(n/N)$ 、縦に p/N 進む (p は1巻きで上がる高さ)。これを巻き数×N 本足せば全長になる。分割を細かくしていくと、なめらかならせんの長さ $\text{巻き数} \times \sqrt{((2\pi r)^2 + p^2)}$ に近づくはず。

結果は **10通りすべて折れ線の式と一致** (差は最大 0.000001)。点の数も 巻き数×N+1 ですべて一致。

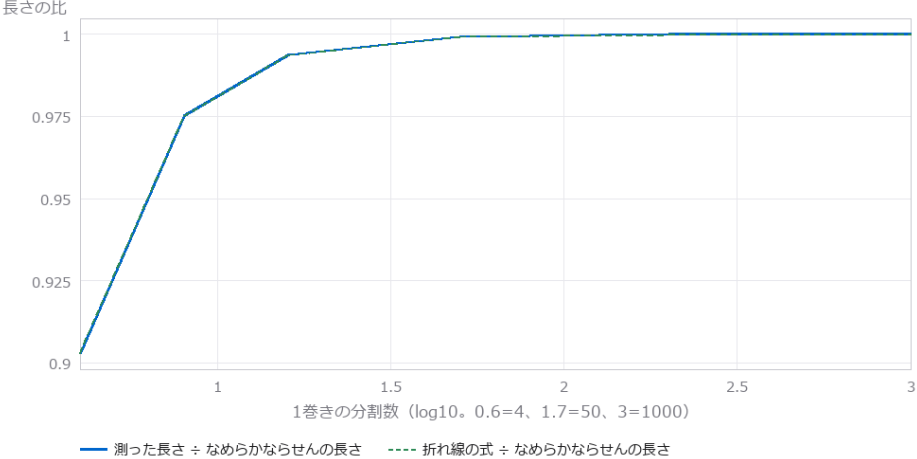
なめらかならせんと比べると、既定の Divisions per Turn = 50 で 0.999358 (0.06% 短い)。4分割なら 0.902912、1000分割なら 0.999998。分割を2倍にすると、足りない分はおよそ1/4になる (8分割で 2.487%、16分割で 0.625%)。

高さ 0 にすると、同じ円を巻き数の回数だけなぞった線になる (4巻きで長さ 25.116 ÷ 円周の4倍)。平らな渦巻きにはならない。渦巻きにしたいときは、半径を変える (Start と End を変える)。

らせんの長さ (折れ線の式・なめらかな式と比べる)

type=Polygon、mode=Turns、半径は Start = End。長さは measure の Perimeter を面ごとに足した値。折れ線の式は $\text{巻き数} \times N \times \sqrt{((2r \cdot \sin(n/N))^2 + (p/N)^2)}$ 。

1巻きの分割を増やすと、らせんの長さは式の値に近づく (半径1・3巻き・高さ3)



分割を増やすと比は1に近づく。実測 (実線) と折れ線の式 (破線) は重なる。

半径	巻き数	高さ	1巻きの分割	点	長さ	折れ線の式	差	なめらかな式との比
1	3	3	4	13	17.233688	17.233688	+0.000000	0.902912
1	3	3	8	25	18.612173	18.612173	-0.000000	0.975133

半径	巻き数	高さ	1巻きの分割	点	長さ	折れ線の式	差	なめらかな式との比
1	3	3	16	49	18.967422	18.967422	+0.000000	0.993746
1	3	3	50	151	19.074549	19.074550	-0.000001	0.999358
1	3	3	200	601	19.086029	19.086030	-0.000001	0.999960
1	3	3	1000	3001	19.086765	19.086765	+0.000000	0.999998
0.5	5	1	50	251	15.729449	15.729450	-0.000000	0.999345
2	2	8	50	101	26.359512	26.359512	-0.000000	0.999403
1	4	0	50	201	25.116209	25.116208	+0.000000	0.999342
0.25	10	5	50	501	16.474695	16.474695	-0.000000	0.999403

わかったこと

- ・折れ線の式に一致。10通りすべてで差は 0.000001 以下。点の数は 巻き数×分割数+1。
- ・既定の50分割で 0.06% 短い。比は 0.999358。ばねや電線の長さを正確に出したいときは、分割を増やすか、式で補正する。
- ・分割2倍で、足りない分は約1/4。弦と弧の差は分割の2乗に反比例して減る。精度を1桁上げるには、分割を約3.2倍にすればよい。
- ・高さ0は渦巻きにならない。半径が一定のままだと、同じ円を何周もなぞるだけ（点も重なる）。

次にやること

- ・半径を変えた渦巻き（Archimedean / Logarithmic）の長さとし
- ・NURBS で出したときの長さ（多角形より長くなるはず。確かめていない）
- ・sweep で太さを付けたときの体積と、らせんの長さ×断面積

attribrandomize の分布は式どおり — Scale Around Middle は標準偏差だった

2026-09-24

何を作ったか

attribrandomize で 99,856 点に乱数を1つずつ入れ、平均・分散・割合を数えて、分布の式から出る値と比べた。10万点あると、平均のぶれは標準偏差の約 1/316 に収まるはず（標準誤差 $\sigma/\sqrt{n}$ ）。

**一様・正規の5通りすべてで、平均のずれは標準誤差の 1.00 倍以内、分散のずれは 0.42% 以内。 ** 式どおりの分布が出ている。

正規分布の **Scale Around Middle は標準偏差**だった。Scale 2 で標準偏差 2.0042、0.5 で 0.4992。画面の名前からは分散なのか幅なのか分からないが、標準偏差として読めばよい。

整数の一様（0～9・刻み1）では、**上端の 9 も出る**。10通りの値がそれぞれ 9.82～10.21%。2つの値（B の確率 0.3）では B が 30.05%。

3成分の「球の中」は、長さの平均 0.7492（式は 3/4）、最大 0.999998。「向き」は長さが全部 1.000000～1.000000 で、平均の向きはほぼ0。

同じ種なら値は1つ残らず同じ。種を 0→1 に変えると、同じ値になった点は 0.0000%。

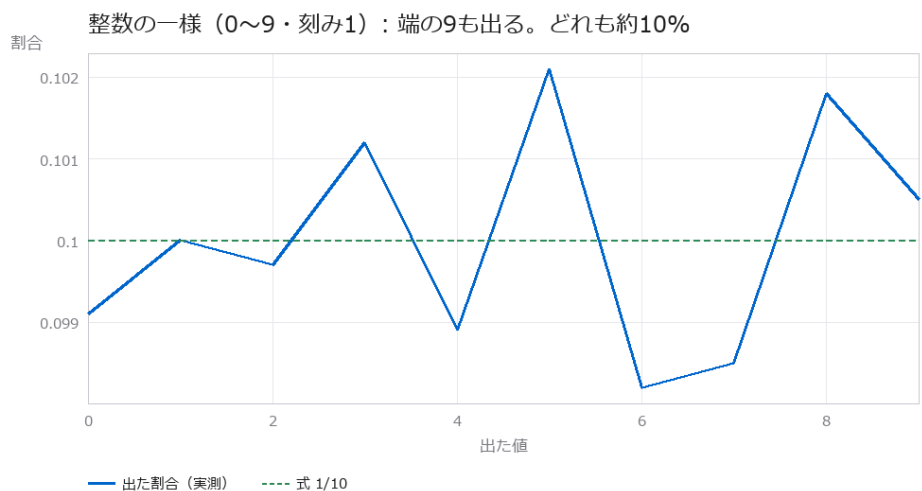
一様・正規の平均と分散（99,856 点）

z は（平均 - 式）÷ 標準誤差。±2 の中なら、偶然のぶれの範囲。

分布	平均	式	z	分散	式	最小	最大
一様 0～1	0.500425	0.5	+0.47	0.083001	0.083333	0.0000	1.0000
一様 2～5	3.500409	3.5	+0.15	0.750799	0.750000	2.0000	5.0000
正規 中央0・Scale 1	0.003162	0	+1.00	0.996149	1.000000	-4.4274	4.1824
正規 中央0・Scale 2	0.005412	0	+0.86	4.016712	4.000000	-8.9782	8.9926
正規 中央3・Scale 0.5	3.000008	3	+0.01	0.249201	0.250000	0.9326	5.1811

整数の一様（0～9・刻み1）の割合

Min 0・Max 9・Step Size 1。



10通りの値がどれも約10%。上端の9も出る。

値	0	1	2	3	4	5	6	7	8	9
割合	9.91%	10.00%	9.97%	10.12%	9.89%	10.21%	9.82%	9.85%	10.18%	10.05%

3成分の分布

長さ = $\sqrt{(x^2+y^2+z^2)}$ 。球の中が1様なら、長さの平均は 3/4。

分布	長さの平均	最小	最大	平均の向き
球の中	0.749162	0.014580	0.999998	[0.000128, -0.001556, 0.001751]
向き	1.000000	1.000000	1.000000	[0.000492, 0.002268, 0.000737]

わかったこと

- ・ 分布は式どおり。1様・正規の5通りで、平均は標準誤差の 1.00 倍以内、分散は 0.42% 以内。
- ・ Scale Around Middle は標準偏差。Scale 2 で標準偏差 2.0042。分散ではない。
- ・ 整数の1様は上端を含む。0~9 なら10通り、どれも約10%。
- ・ 球の中は、中まで均一。長さの平均 0.7492 (式 0.75) 。表面だけに散る「向き」とは別物。
- ・ 同じ種なら、同じ値。種を変えれば全部の点が変わる。
- ・ 速い。99,856 点で 0.01~0.13 秒 (最初の1回は読み込みを含むので遅い) 。

次にやること

- ・ Exponential・Log-Normal の Middle Value と Spread Around Middle の意味
- ・ Use Seed Attribute で id を種にしたとき、点を消しても値が変わらないか
- ・ Cone Angle を付けた向きの分布が、角度どおりに収まるか

extractcentroid の Center of Mass

は、閉じていれば体積・開いていれば面積の中心

2026-09-24

何を作ったか

「中心」には何通りもある。点の平均、囲む箱の中心、面積の中心、体積の中心。形によっては全部ちがう値になる。答えが計算で出る4つの形を作り、extractcentroid の Method ごとに出てくる値を式と突き合わせた。

****Center of Mass** は、閉じた形なら体積の中心、開いた形なら面積の中心。******
四角錐（閉じている）では $y=0.75$ （体積の中心 $3/4$ ）、底を抜くと $y=1$ （側面の面積の中心）。L字の板では $(0.833333, 0.833333)$ （面積の中心 $5/6$ ）。点の平均ではない。

****Bounding Box Center** は、囲む箱の真ん中。****** 四角錐なら $y=1.5$ で、形の重さのある所からいちばん離れる。

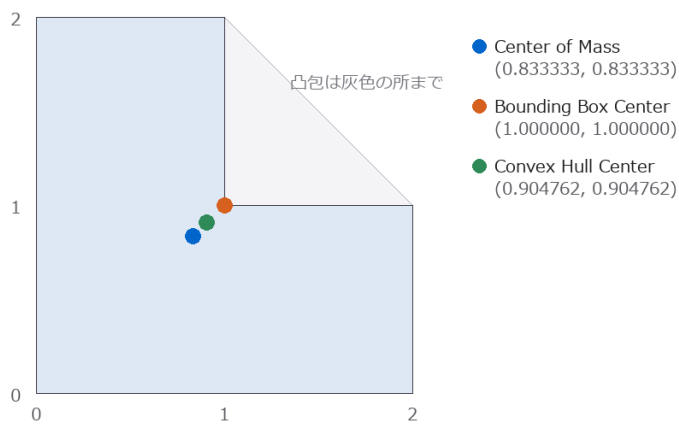
****Convex Hull Center** は、凸包（へこみを埋めた外形）の体積の中心。****** L字の柱では $(0.904762, \dots)$ で、式 $19/21$ と一致。底を抜いた四角錐でも、凸包は底をふさいだ形になるので $y=0.75$ 。平らな L字の板では凸包に体積が無く、 $(1, 1)$ が出た（点の平均とも箱の中心とも同じ値なので、どちらに倒れたのかは確かめていない）。

Piece Elements を Primitive にしても Point にしても、結果は同じ。

形と Method ごとの中心（式と一致したものを右に書く）

Run Over = Detail（形全体で1つ）、Output = Points。四角錐は底 2×2 ・高さ3、L字は 2×2 から 1×1 の角を欠いたもの、柱は高さ1。

L字の柱を上から見た図。3つの Method で、中心の置き場所がちがう



L字の柱を上から。Center of Mass（青）は本当の体積の中心、Convex Hull（緑）は灰色まで含めた外形の中心、Bounding Box（橙）は箱の真ん中。

形	Method	出た中心	一致した式
L字の板（開いた面1枚）	Center of Mass	$(0.833333, 0, 0.833333)$	面積の中心
L字の板（開いた面1枚）	Bounding Box Center	$(1, 0, 1)$	点の平均・箱の中心
L字の板（開いた面1枚）	Convex Hull Center	$(1, 0, 1)$	点の平均・箱の中心

形	Method	出た中心	一致した式
四角錐（閉じている）	Center of Mass	(-0, 0.75, 0)	体積の中心
四角錐（閉じている）	Bounding Box Center	(0, 1.5, 0)	箱の中心
四角錐（閉じている）	Convex Hull Center	(0, 0.75, -0)	体積の中心
底の無い四角錐（開いている）	Center of Mass	(0, 1, -0)	面積の中心
底の無い四角錐（開いている）	Bounding Box Center	(0, 1.5, 0)	箱の中心
底の無い四角錐（開いている）	Convex Hull Center	(0, 0.75, -0)	凸包の体積の中心
L字の柱（閉じている）	Center of Mass	(0.833333, 0.5, 0.833333)	体積の中心
L字の柱（閉じている）	Bounding Box Center	(1, 0.5, 1)	点の平均・箱の中心
L字の柱（閉じている）	Convex Hull Center	(0.904762, 0.5, 0.904762)	凸包の体積の中心

わかったこと

- ・ Center of Mass は「形の中身」の中心。閉じていれば体積、開いていれば面積で重みを付ける。点の平均ではない（点が片寄った形では試していない）。
- ・ Bounding Box Center は、先が尖った形ほど外れる。四角錐で $y=1.5$ （体積の中心 0.75 の2倍の高さ）。
- ・ Convex Hull Center は、へこみを埋めた形の中心。L字の柱で $19/21 \approx 0.905$ 。開いた形でもふさいだ形として扱う。
- ・ Primitive / Point の切り替えは、Detail で回すときは結果を変えない。

次にやること

- ・ 点がかたよった形（片側だけ細かく割った箱）でも Center of Mass が動かないか
- ・ Run Over = Pieces で、破片ごとの中心が1つずつ正しく出るか
- ・ RBD の重心（center of mass）と extractcentroid の値が一致するか

triangulate2d の三角形は $2n-2-h$ 枚 — 9通りすべて式どおり、面積は凸包と一致

2026-09-24

何を作ったか

平面の点 n 個を三角形で埋めると、外周（凸包）に乗る点が h 個なら、三角形は $2n - 2 - h$ 枚になる。オイラーの多面体定理から出る式で、並べ方によらない。面を張った範囲は凸包になるので、面積の合計も凸包の面積と同じはず。

ばらまいた点（scatter、10～100,000点）と、格子に並んだ点（ $3\times 3\sim 51\times 51$ ）で確かめた。凸包と面積は Python 側で別に計算した。

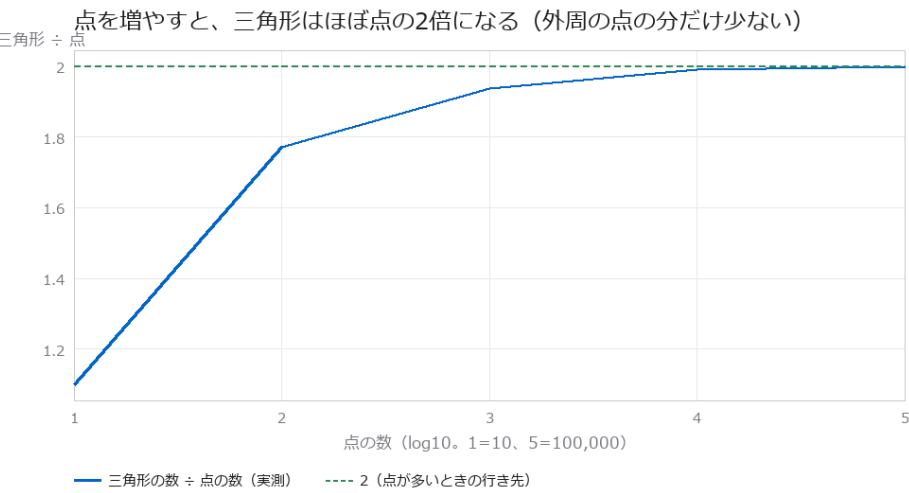
9通りすべてで三角形の数が式と一致（例外なし）。面積の差は最大 0.000001。三角形以外の面は1枚も出なかった。

格子の点は、4点が同じ円に乗るのでどちらの対角線で割っても成り立つが、それでも枚数は $2(k-1)^2$ で、1マスを2枚に割っている。

速さは、100,000 点で 0.62 秒（10,000 点で 0.059 秒）。1万点から10万点に増やすと、時間は約10倍。

点の数と三角形の数

scatter は 4×3 の板の上。格子は grid の面を消して点だけにしたもの。 h は凸包に乗る点の数（一直線上の点も数える）。



外周の点の割合が減るので、三角形 ÷ 点 は2に近づく。

点の置き方	n	h	三角形	$2n-2-h$	面積	凸包の面積	秒
scatter 10	10	7	11	11	6.813466	6.813466	0.0009
scatter 100	100	21	177	177	11.764791	11.764791	0.0012
scatter 1000	1,000	61	1,937	1,937	11.858296	11.858296	0.0122
scatter 10000	10,000	98	19,900	19,900	11.995832	11.995833	0.0586

点の置き方	n	h	三角形	$2n-2-h$	面積	凸包の面積	秒
scatter 100000	100,000	207	199,791	199,791	11.999218	11.999218	0.6184
格子 3x3	9	8	8	8	100.000000	100.000000	0.0002
格子 5x5	25	16	32	32	100.000000	100.000000	0.0002
格子 11x11	121	40	200	200	100.000000	100.000000	0.0004
格子 51x51	2,601	200	5,000	5,000	100.000001	100.000000	0.0091

わかったこと

- ・ 三角形は $2n-2-h$ 枚。9通りすべて一致。点の数と外周の点の数だけで、分ける前に枚数が分かる。
- ・ 面を張る範囲は凸包。面積の差は最大 0.000001。へこんだ形の中だけを埋めたいときは、拘束の辺か Silhouette を渡す必要がある（今回は試していない）。
- ・ 格子でも崩れない。同じ円に4点が乗る並びでも、1マス2枚ずつになる。
- ・ 10万点で約0.6秒。1万点→10万点で約10倍。

次にやること

- ・ Silhouette と拘束の辺で、へこんだ形の中だけを埋めたときの枚数
- ・ Cap Maximum Area で三角形を足したときの枚数と面積
- ・ ドロネーの条件（外接円に他の点が入らない）を数で確かめる

groupexpand の1段は「隣」の決め方しだい 一点と辺共有は菱形、面の既定は正方形

2026-09-24

何を作ったか

格子の真ん中の1つから groupexpand で広げ、段ごとの数を数えた。辺でつながった隣へ広がるなら k 段で菱形 $2k^2+2k+1$ 個、角を共有する隣まで広がるなら正方形 $(2k+1)^2$ 個になる。

結果は、点のグループが **菱形**、面で Require Primitives Share Edge を入れると **菱形**、切ったまま（既定）だと **正方形**。7 通りの段数（0～8）で、すべて式の数ちょうどだった。

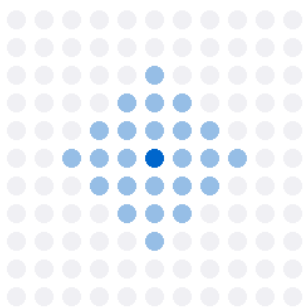
つまり、面のグループを既定のまま広げると、斜めの隣にも一度に広がる。8段で 289 枚と、辺だけのとき（145 枚）の約2倍になる。

負の段数は縮める。9×9 の点の塊から -1 段で 7×7、-2 段で 5×5、-4 段で 1 点（すべて式どおり）。1段ごとに外周が1列ずつ削られる。

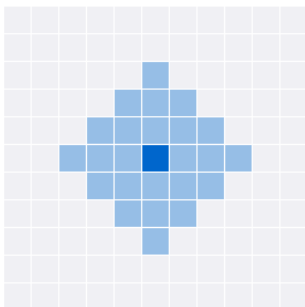
広げた段数と数

点は 21×21 の格子の真ん中の点から、面は 21×21 枚の真ん中の面から。

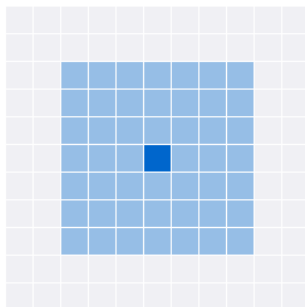
真ん中の1つから 3 段広げた範囲



点のグループ: 25 個



面・辺を共有: 25 個



面・角を共有（既定）: 49 個

3段広げたときの範囲。面の既定は斜めにも広がる。

段数	点のグループ	面（辺を共有する隣だけ）	面（角を共有する隣まで・既定）	菱形の式	正方形の式
0	1	1	1	1	1
1	5	5	9	5	9
2	13	13	25	13	25
3	25	25	49	25	49
4	41	41	81	41	81
6	85	85	169	85	169
8	145	145	289	145	289

縮める（負の段数）

9×9 の点の塊から。

段数	数	式
0	81	81
-1	49	49
-2	25	25
-4	1	1

わかったこと

- ・点は辺でつながった隣へ。k 段で $2k^2+2k+1$ 個の菱形。
- ・面の既定は、角を共有する隣まで。k 段で $(2k+1)^2$ 個の正方形。辺でつながった隣だけにしたいときは Require Primitives Share Edge を入れる。
- ・負の段数は、外周を1列ずつ削る。 $9\times 9 \rightarrow 7\times 7 \rightarrow 5\times 5$ 。
- ・速い。 21×21 の格子なら、どの段数でも1ミリ秒未満（最初の1回を除く）。

次にやること

- ・ Restrict by Normal Spread Angle で、箱の角を越えずに止められるか
- ・ Flood Fill と connectivity の結果が同じになるか
- ・ 三角形の網や、ばらつきのある網での広がり方

copyxform は「1回分をくり返す」ではなく「値を i 倍して1回かける」 — 回転 + 移動は輪にならない

2026-09-24

何を作ったか

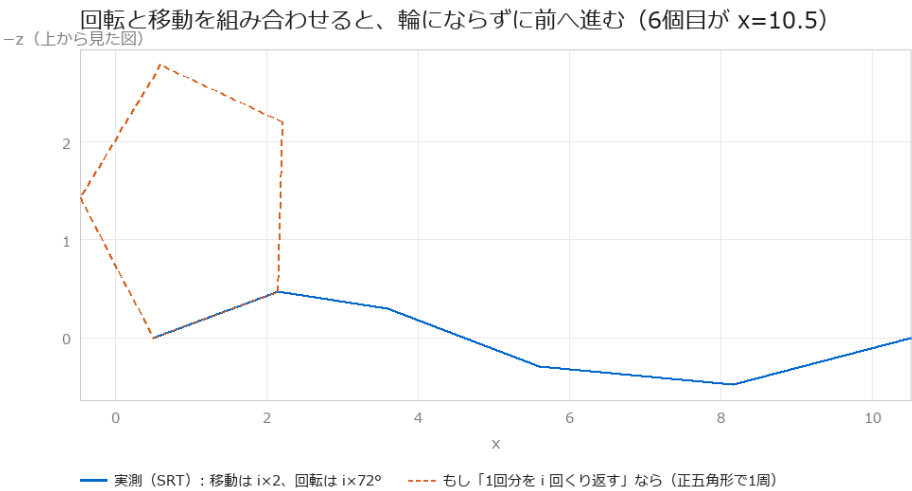
copyxform (Copy and Transform) は、1つ置くたびに同じ変形を重ねていく箱だと思われがち。もしそうなら、i 個目は「1回分の変形 M を i 回かけたもの (M^i)」になる。ところが、もう1つの読み方がある。**移動を i 倍・回転を i 倍・拡大を i 乗した値で、1回だけ変形する**。移動だけ・拡大だけなら2つは同じ答えになるが、回転と移動を組み合わせると大きく分かれる。4通りで確かめた。

結果は **4通りすべてで「値を i 倍して1回」の式と一致** (ずれは最大 0.000001)。「i 回くり返す」の式とは、回転を含む2通りで $10.000 \cdot 10.000$ ずれた。

たとえば回転 72° + 移動 2 を6個並べると、5回で1周するはずなのに、6個目は $x=10.5$ にいる。移動 $2 \times 5 = 10$ と、 360° 回った元の位置 0.5 の和。輪 (正五角形) にはならず、蛇行しながら x の方へ進む。拡大は i 乗だった ($0.8 \rightarrow 0.64 \rightarrow 0.512 \dots$)。i 倍 ($0.8 \rightarrow 0.6 \rightarrow 0.4$) ではない。

2つの読み方と、実測とのずれ (各コピーの中心の最大距離)

元の形は $x=0.5$ に置いた一辺 0.2 の箱。中心は箱の8点の平均。



青が実測。輪になるのは「くり返す」読み方 (橙の破線) のとき。

置き方	個数	値を i 倍して1回	1回分を i 回くり返す
移動だけ ($x+1$)	5	0.000000	0.000000
拡大だけ (0.8倍)	5	0.000000	0.000000
回転 72° + 移動2 (SRT)	6	0.000001	10.000000
回転 72° + 移動2 (TRS)	6	0.000001	10.000000

回転 72° + 移動2 (SRT) の各コピーの中心

i 個目 = 拡大 $\rightarrow$ 回転 $i \times 72^\circ \rightarrow$ 移動 $i \times 2$ 。

i	x	z
0	0.500000	0.000000
1	2.154508	-0.475528
2	3.595492	-0.293893
3	5.595492	0.293893
4	8.154509	0.475528
5	10.500000	0.000000

わかったこと

- ・ 値を i 倍して1回かける。移動は i 倍、回転は i 倍、拡大は i 乗。それを Transform Order の順で1回かける。
- ・ 回転と移動を混ぜると輪にならない。 $72^\circ \times 5$ で1周しても、移動は $2 \times 5 = 10$ 進んでいる。輪に並べたいときは、Pivot を輪の中心に置くか、copy to points で円周上の点に置く（どちらも今回は試していない）。
- ・ 回転の軸と同じ向きの移動なら、らせん階段になる。軸方向の移動は回転と入れ替えても同じなので、2つの読み方が一致する（式から言えること。今回は測っていない）。
- ・ 拡大は i 乗。0.8 なら 5個目で $0.8^4 \approx 0.41$ 倍。

次にやること

- ・ Pivot Translate を輪の中心に置いたとき、正五角形に並ぶか
- ・ 回転の軸 (y) に沿った移動で、らせん階段が式どおりになるか
- ・ copy to points との速さの差 (10万個)

pointjitter の Scale は「幅」 — 各軸 $-s/2 \sim +s/2$ の一様で、箱の中に散る

2026-09-24

何を作ったか

99,856 点に pointjitter をかけ、元の位置からの動きを軸ごとに数えた。Scale が「動く幅」なら動きは $-s/2 \sim +s/2$ の一様で分散は $s^2/12$ 、「半径」なら $-s \sim +s$ で分散は $s^2/3$ になる。

**Scale は幅だった。 ** Scale 1 で動きは各軸 $-0.500 \sim +0.500$ 、分散は $\times 0.08336$ (式 $1/12 = 0.08333$)。 Scale $0.1 \cdot 1 \cdot 2$ の3通り $\times 3$ 軸で、分散のずれは 0.88% 以内。

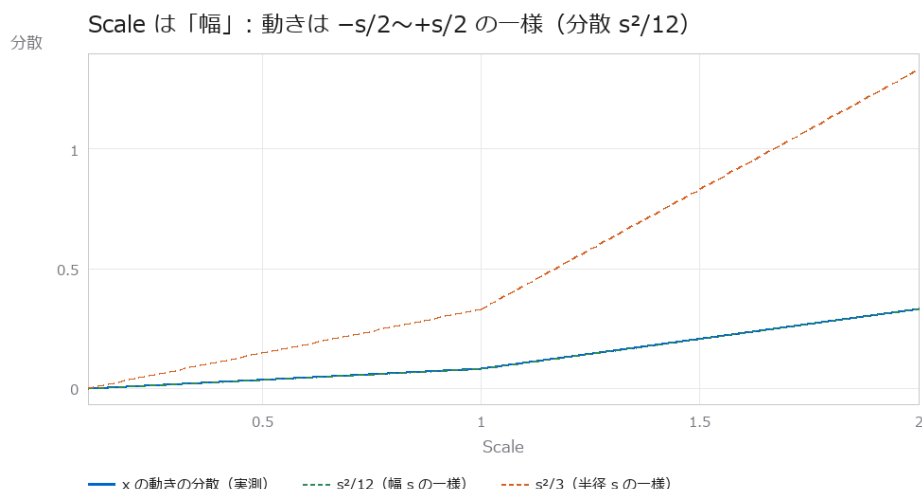
**散る範囲は球ではなく箱。 ** 動いた距離の最大は 0.8516 で、軸の端 0.5 を超えて、箱の角 $0.5 \times \sqrt{3} \div 0.866$ の近くまで届いた。

Axis Scales は軸ごとの倍率。(1, 0, 0.5) にすると y は1点も動かず、z は $-0.250 \sim 0.250$ (幅が半分)。x の動きは Axis Scales を触らないときと1点残らず同じで、同じ乱数に倍率を掛けているだけだと分かる。

Seed を 1 $\rightarrow$ 2 に変えると、x の動きが同じだった点は 0%。

Scale と動きの範囲・分散 (99,856 点)

動き = jitter 後の位置 - 元の位置。



実測は $s^2/12$ の線に乗る。 $s^2/3$ (半径の読み方) とは4倍ちがう。

設定	x の範囲	y の範囲	z の範囲	x の分散	y の分散	z の分散	距離の最大
Scale 0.1	-0.0500 $\sim$ +0.0500	-0.0500 $\sim$ +0.0500	-0.0500 $\sim$ +0.0500	0.000834	0.000826	0.000835	0.0852
Scale 1	-0.5000 $\sim$ +0.5000	-0.5000 $\sim$ +0.5000	-0.5000 $\sim$ +0.5000	0.083362	0.082599	0.083459	0.8516
Scale 2	-0.9999 $\sim$ +1.0000	-1.0000 $\sim$ +1.0000	-0.9999 $\sim$ +0.9999	0.333446	0.330397	0.333837	1.7031
Scale 1 $\cdot$ Axis (1, 0, 0.5)	-0.5000 $\sim$ +0.5000	+0.0000 $\sim$ +0.0000	-0.2500 $\sim$ +0.2500	0.083362	0.000000	0.020865	0.5578

設定	x の範囲	y の範囲	z の範囲	x の分散	y の分散	z の分散	距離の最大
Scale 1・Seed 2	-0.5000～+ 0.5000	-0.5000～+ 0.5000	-0.5000～+ 0.5000	0.083350	0.083062	0.082991	0.8599

わかったこと

- ・ Scale は動く幅。各軸 $-s/2 \sim +s/2$ 。0.05 だけ揺らしたいなら Scale 0.1。
- ・ 箱の中に散る。角の方向には $\sqrt{3}$ 倍（約1.73倍）遠くまで動く。距離をそろえたいなら、attribrandomize の Inside Sphere で動きを作って足す（実験106）。
- ・ Axis Scales は同じ乱数への倍率。0 にした軸は動かない。
- ・ 速い。10万点で 0.04 秒（最初の1回を除く）。

次にやること

- ・ Use Point Scale を入れたとき、pscale が幅に掛かるのか
- ・ Mask に 0.5 を入れたとき、幅が半分になるのか、半分の点だけ動くのか
- ・ Use ID Attribute で点を消しても、残った点の動きが変わらないか

vdbcombine の和・積・差は球2つの式に収束 — ボクセル 0.0125 で 0.08% 以内、どれも少なめ

2026-09-24

何を作ったか

半径1の球を2つ、中心の距離 1 で重ね、vdbcombine の SDF Union / Intersect / Difference で合わせてから多角形に戻し、体積を式と比べた。重なり（レンズ）の体積は $\pi(4r+d)(2r-d)^2/12$ で出せるので、和・積・差の答えはすべて式で分かる。

**3つとも、ボクセルを細かくするほど式に近づいた。* ボクセル 0.0125 で、和 -0.034%・積 -0.076%・差 -0.039%。ボクセル 0.1 では 和 -0.95%・積 -2.14%。

ずれはすべてマイナス（体積が少なめ）。 実験094（VDB は形をやせさせる）と同じ向き。ずれの割合は積（レンズ）が一番大きい。レンズは縁が鋭く尖っていて、その先をボクセルが丸めるからだと考えられる（尖りの影響だけを分けては確かめていない）。

ボクセルを半分にしたときの減り方は、最初は約1/4（0.1→0.05 で和が3.7分の1）、細くなるほど鈍る。元の球（200×200 の多角形）自体が -0.023% 小さいので、そこが底になる。

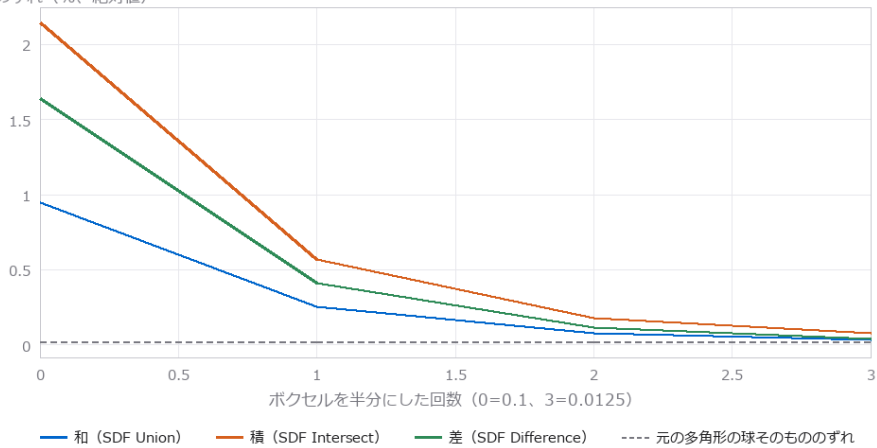
速さ: ボクセル 0.0125 で和が 1.9 秒（VDB 化・合成・多角形化・体積の合計）。半分にすると約3〜4倍。

つまずいた点: 素性の控え（node_dump.py）では Operation の選択肢が12個で切れていて、SDF Intersect / Difference が載っていないかった。本物は18個ある。控えを直した（選択肢は全部残す）。

ボクセルの大きさと体積（式との比）

球は polymesh 200×200。vdbfrompolygons → vdbcombine → convertvdb (Polygons) → measure の Volume を面ごとに足した値。

ボクセルを半分にするたびに、式とのずれが減る（どれも体積は少なめに出る）
式とのずれ（%、絶対値）



どの合わせ方も、ボクセルを細かくすると式に近づく。灰色の破線は、元の多角形の球が持っているずれ。

ボクセル	合わせ方	体積	式	ずれ	秒
0.1	和 (SDF Union)	7.001569	7.068583	-0.948%	0.097
0.1	積 (SDF Intersect)	1.280965	1.308997	-2.142%	0.014

ボクセル	合わせ方	体積	式	ずれ	秒
0.1	差 (SDF Difference)	2.832658	2.879793	-1.637%	0.022
0.05	和 (SDF Union)	7.050587	7.068583	-0.255%	0.161
0.05	積 (SDF Intersect)	1.301543	1.308997	-0.570%	0.043
0.05	差 (SDF Difference)	2.867955	2.879793	-0.411%	0.077
0.025	和 (SDF Union)	7.063037	7.068583	-0.079%	0.516
0.025	積 (SDF Intersect)	1.306632	1.308997	-0.181%	0.160
0.025	差 (SDF Difference)	2.876511	2.879793	-0.114%	0.312
0.0125	和 (SDF Union)	7.066169	7.068583	-0.034%	1.903
0.0125	積 (SDF Intersect)	1.308006	1.308997	-0.076%	0.634
0.0125	差 (SDF Difference)	2.878679	2.879793	-0.039%	1.266

わかったこと

- ・和・積・差は式どおりに収束する。ボクセル 0.0125 で3つとも 0.08% 以内。
- ・体積は少なめに出る。12通りすべてマイナス。ぴったり合わせたいときは、細かくするか、少し太らせて戻す。
- ・尖った所を含む形ほどずれる。同じボクセルでも、積（レンズ）は和の約2倍ずれた。
- ・Operation の選択肢は18個。SDF Intersect・SDF Difference・Topology 系は後ろのほうにある。

次にやること

- ・Topology Union / Intersect（形ではなく、ボクセルの有無で合わせる）の結果
- ・polygon の boolean（実験098）と、同じ球2つで速さと精度を比べる
- ・Half Width を広げると、細い所のやせ方が変わるか

timeblend は速度が無ければ直線、あれば F^2 を誤差0でつなぐ — v は「1秒あたり」

2026-09-24

何を作ったか

点を1つ、高さ $y = F^2$ (F はフレーム) で動かし、timeblend で整数フレームの間 (1.25・1.5・3.3 など) を作らせた。整数フレームの形だけから直線でつなぐなら、1.5 フレームでは $(1+4)/2 = 2.5$ になる (本当は 2.25) 。

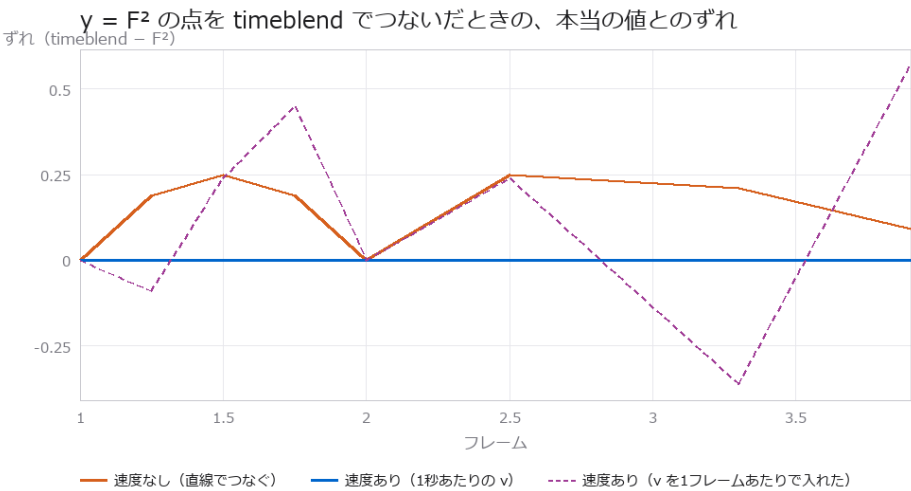
****速度が無いと、直線でつなぐ。* 8通りすべてで直線の式と一致 (差 0.000001) 。本当の値からは最大 0.250 ずれた。**

****速度 v を持たせて Use Velocity When Interpolating Position を入れると、 F^2 をぴったり再現した** (ずれ最大 0.000001) 。両端の位置と速度を使う3次のつなぎ方 (エルミート補間) なら2次式は誤差0になるので、それと合う。**

**** v は1秒あたりの速さとして読まれる。* 2F×24 (fps) を入れたときに合った。fps を掛け忘れて1フレームあたりの値 (2F) を入れると、ずれは最大 0.569 で、速度なし (0.250) より悪くなることもある。**

フレームと高さ ($y = F^2$)

fps = 24。 v は attribwrangle で毎フレーム入れた。 Hold First Frame は切った。



青 (1秒あたりの v) はずれ0。橙 (速度なし) は整数フレームの間でふくらむ。

フレーム	本当の値 F^2	速度なし	直線の式	速度あり	v を1フレームあたり
1	1.000000	1.000000	1.000000	1.000000	1.000000
1.25	1.562500	1.750000	1.750000	1.562500	1.472656
1.5	2.250000	2.500000	2.500000	2.250000	2.489583
1.75	3.062500	3.250000	3.250000	3.062500	3.511719
2	4.000000	4.000000	4.000000	4.000000	4.000000
2.5	6.250000	6.500000	6.500000	6.250000	6.489583

フレーム	本当の値 F^2	速度なし	直線の式	速度あり	v を1フレームあたり
3.3	10.890000	11.099999	11.100000	10.889999	10.527749
3.9	15.210000	15.299999	15.300000	15.209999	15.779249

わかったこと

- 速度が無ければ直線。曲がって動くものは、フレームの間に近道する (F^2 では最大 0.25)。
- 速度を渡せば、2次の動きは誤差0。Use Velocity When Interpolating Position を入れる。
- v は1秒あたり。24 fps なら、1フレームあたりの動き×24。間違えると、速度なしより悪くなる。
- 今回は点に $id=0$ を入れた。 id が無い場合は試していない。

次にやること

- id が無い／点の数がフレームで変わるとき、timeblend はどうするか
- 回転 (orient) を Smoothly Interpolate で補うと、角度はどうつながるか
- モーションブレンダー (サブフレームのレンダ) に timeblend を挟んだときの見た目

sweep の管は曲がり角で細る — 不足は $1-\cos(\text{角}/2)$ 、Stretch Around Turns で $A \times L$ に一致

2026-09-24

何を作ったか

sweep の Round Tube で太さ 0.1 の管を作り、体積を「断面積 × 長さ」と比べた。断面は円ではなく k 角形 (Columns) なので、断面積は $(k/2) \cdot a^2 \cdot \sin(2\pi/k)$ 。端は Single Polygon でふさいだ。

****まっすぐな線では、ぴったり 断面積×長さ**** (4通りすべて差 0.000000)。

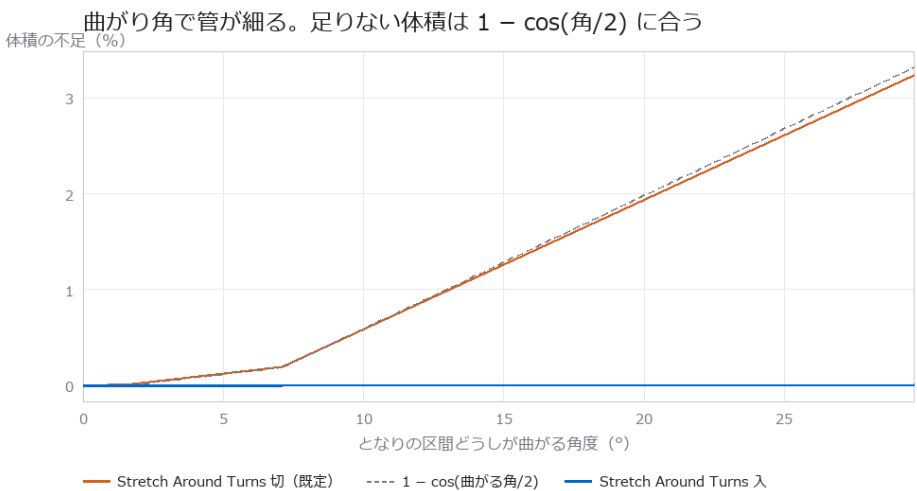
****らせん (実験105) に沿わせると、既定では体積が足りない。* 1巻き50分割で -0.191%、12分割で -3.24%。折れ線の曲がり角で、断面が角の二等分の向きに置かれ、そのぶん管が細るためと考えて、不足を $1 - \cos(\text{曲がる角}/2)$ と比べると、12分割 (角 29.6°) で式 3.32% に対し実測 3.24%、50分割 (7.11°) で式 0.192% に対し 0.191%。近いが完全には一致しない。**

****Stretch Around Turns を入れると、曲がり角の断面を引き伸ばして太さを保つ。* すべての場合で断面積×長さ との差が 0.038% 以内になった。**

Columns (8 か 32) を変えても、不足の割合はほとんど変わらない。

道の形・断面の角数・Stretch Around Turns と体積

管の半径 0.1。らせんは半径1・3巻き・高さ3 (実験105と同じ)。長さは道の折れ線の長さ (measure of the Perimeter)。



既定 (橙) は角が大きいほど体積が足りない。Stretch Around Turns を入れる (青) とほぼ0。

道	角数	Stretch	曲がる角	体積	断面積×長さ	ずれ	$1-\cos(\text{角}/2)$
直線 5	8	切	0.00°	0.141421	0.141421	+0.000%	+0.000%
直線 5	8	入	0.00°	0.141421	0.141421	+0.000%	+0.000%
直線 5	32	切	0.00°	0.156072	0.156072	-0.000%	+0.000%
直線 5	32	入	0.00°	0.156072	0.156072	-0.000%	+0.000%
らせん 50	8	切	7.11°	0.538476	0.539510	-0.192%	-0.192%

道	角数	Stretch	曲がる角	体積	断面積×長さ	ずれ	$1 - \cos(\text{角}/2)$
らせん 50	8	入	7.11°	0.539507	0.539510	-0.001%	-0.192%
らせん 50	32	切	7.11°	0.594263	0.595402	-0.191%	-0.192%
らせん 50	32	入	7.11°	0.595401	0.595402	-0.000%	-0.192%
らせん 200	8	切	1.78°	0.539770	0.539834	-0.012%	-0.012%
らせん 200	8	入	1.78°	0.539834	0.539834	-0.000%	-0.012%
らせん 200	32	切	1.78°	0.595688	0.595760	-0.012%	-0.012%
らせん 200	32	入	1.78°	0.595760	0.595760	+0.000%	-0.012%
らせん 12	8	切	29.61°	0.516439	0.533863	-3.264%	-3.320%
らせん 12	8	入	29.61°	0.533659	0.533863	-0.038%	-3.320%
らせん 12	32	切	29.61°	0.570103	0.589170	-3.236%	-3.320%
らせん 12	32	入	29.61°	0.589116	0.589170	-0.009%	-3.320%

わかったこと

- ・まっすぐなら 断面積×長さ ちょうど。断面は k 角形なので、円の面積 πa^2 ではない（8角形なら 0.90 倍）。
- ・既定のままだと、曲がり角で管が細る。体積の不足はほぼ $1 - \cos(\text{角}/2)$ 。1巻き12分割のらせんで約3%。
- ・Stretch Around Turns を入れれば太さが保たれる。パイプや電線の太さをそろえたいときは入れる。
- ・道を細かく割っても不足は減る。角が半分になると不足は約1/4。

次にやること

- ・Max Stretch を下げたとき、鋭い角でどこまで太さが保たれるか
- ・Square Tube / Ribbon の面積と体積
- ・道に pscale を持たせたとき、太さが点ごとに変わるか

attribfill の到着時間は「網の辺をたどる道のり」 — 四角の網ではマンハッタン距離そのもの

2026-09-24

何を作ったか

attribfill で値を埋め、式で出る答えと比べた。

**Interpolate (Poisson) は式どおり。 ** 4×2 の板の左端を0・右端を1に固定すると、中は x に比例する直線で埋まった (3通りの細かさで、ずれ 0.000000) 。

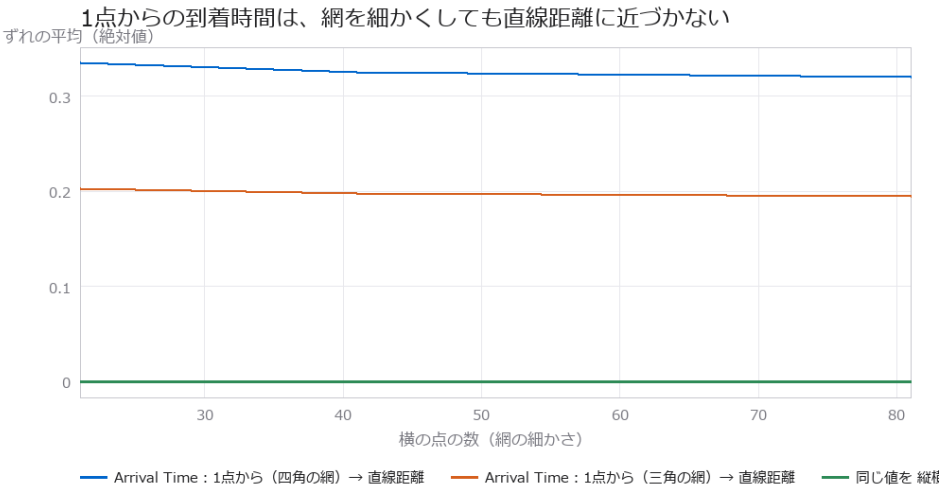
**Arrival Time (Eikonal) も、網の辺と同じ向きに進むなら式どおり。 **
左端から出発すると、到着時間は x+2 にぴったり。球の北極から出発すると、弧の長さ $r \cdot \theta$ に対して ずれ 0.0024 (24段) → 0.00014 (96段) で、細かくすると約1/4ずつ減った (経線がそのまま辺になっている) 。

ところが1点から出発すると、直線距離にならない。 ** 四角の網では、到着時間は **縦横の道のり $|x|+|z|$ と1点残らず一致した (ずれ 0.000000) 。直線距離とは、角で最大 0.7639 (= $3 - \sqrt{5}$) ずれる。網を 21→81 と細かくしても、ずれの平均は 0.335 → 0.320 でほとんど変わらない。網の辺をたどる最短の道のりを数えているからと考えられる。

三角の網 (斜めの辺がある) にすると、ずれの平均は 0.195 に下がるが、最大は 0.7639 のまま (斜めの辺が向いていない側の角) 。

埋め方と、式とのずれ

板は 4×2 (横の点 21・41・81) 。球は半径1の polymesh (段 24・48・96) 。固定する点は Boundary Group に入れた。



1点から出発すると、網を細かくしてもずれが減らない。縦横の道のり (緑) とは0。

埋め方と答え	細かさ	ずれの平均	ずれの最大
Interpolate : 左0・右1 → 直線	21	0.000000	0.000000
Arrival Time : 左端から → x	21	0.000000	0.000000
Arrival Time : 1点から (四角の網) → 直線距離	21	0.334572	0.763932

埋め方と答え	細かさ	ずれの平均	ずれの最大
同じ値を 縦横の道のり $ x + z $ と比べる	21	0.000000	0.000000
Arrival Time : 1点から (三 角の網) → 直線距離	21	0.202707	0.763932
Interpolate : 左0・右1 → 直線	41	0.000000	0.000000
Arrival Time : 左端から → x	41	0.000000	0.000000
Arrival Time : 1点から (四 角の網) → 直線距離	41	0.325229	0.763932
同じ値を 縦横の道のり $ x + z $ と比べる	41	0.000000	0.000000
Arrival Time : 1点から (三 角の網) → 直線距離	41	0.198002	0.763932
Interpolate : 左0・右1 → 直線	81	0.000000	0.000000
Arrival Time : 左端から → x	81	0.000000	0.000000
Arrival Time : 1点から (四 角の網) → 直線距離	81	0.319703	0.763932
同じ値を 縦横の道のり $ x + z $ と比べる	81	0.000000	0.000000
Arrival Time : 1点から (三 角の網) → 直線距離	81	0.194997	0.763932
Arrival Time : 球の北極から → 弧の長さ	24	0.001221	0.002442
Arrival Time : 球の北極から → 弧の長さ	48	0.000292	0.000585
Arrival Time : 球の北極から → 弧の長さ	96	0.000072	0.000143

わかったこと

- ・ Interpolate は、まわりの平均で埋める。両端を固定した板なら直線になる。
- ・ Arrival Time は「辺をたどる道のり」。四角の網では $|x|+|z|$ 。丸く広がる波にはならない。
- ・ 細かくしても直らない。ずれは網の向きで決まるので、点を増やしても同じ。直線距離がほしいなら、点どうしの距離を VEX で直接測る (distance や xyzdist) 。
- ・ 網の向きに沿った広がりなら正確。端からの距離、球の経線に沿った距離は式どおり。

次にやること

- ・ Blur (Diffusion) の Diffusion Time と、ぼけ幅 (ガウスの広がり) の関係
- ・ remesh で向きのばらけた三角の網にすると、1点からの到着時間は丸くなるか
- ・ Speed Attribute を場所で変えたとき、屈折のように曲がるか

uvflatten は円筒をゆがみ0.0001%で開く — 半球は SCP 13%・ABF 10% ゆがむ

2026-09-24

何を作ったか

49×17 点の板を丸めて、円筒（半径1・高さ2）と半球を作り、uvflatten で開いた。辺ごとに「UV の長さ ÷ 3D の長さ」を出し、そのばらつき（標準偏差 ÷ 平均）をゆがみとした。ゆがみが無ければ、どの辺でも比は同じになる。

**円筒は、ゆがまずに開いた。 ** ばらつきは SCP で 0.0001%、ABF で 0.0021%。開いた長方形の縦横比は 3.139349 で、式（48角形の周 ÷ 高さ） 3.139350 と6桁一致した。円筒は切り開けば平らになる形なので、これが正解。

**半球は、どちらの開き方でもゆがむ。 ** ばらつきは SCP で 13.5%、ABF で 10.4%。球の一部は、どう開いても長さを保てない（地図の図法と同じ問題）。ABF のほうが今回は小さかった。

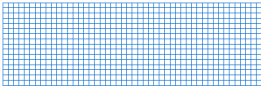
UV は横幅がほぼ 1（0.999893）になるように縮められていた。

形・開き方と、ゆがみ

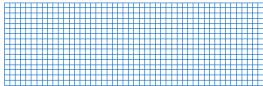
ゆがみ = 辺ごとの（UV の長さ ÷ 3D の長さ）の標準偏差 ÷ 平均。円筒は板を丸めたので、最初から1本の切れ目がある。

開いた UV（同じ網を、形と開き方を変えて）

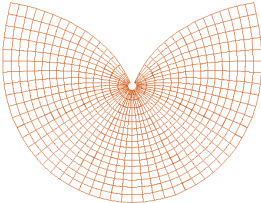
円筒 × Spectral (SCP) ゆがみ 0.00%



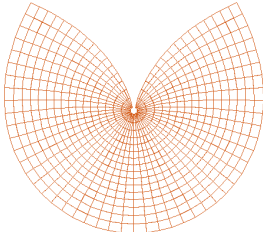
円筒 × Angle-Based (ABF) ゆがみ 0.00%



半球 × Spectral (SCP) ゆがみ 13.49%



半球 × Angle-Based (ABF) ゆがみ 10.45%



円筒（青）はきれいな長方形。半球（橙）は扇形に開き、場所によって辺の長さの比がそろわない。

形	開き方	ゆがみ	UV の幅	UV の高さ	縦横比	秒
円筒	Spectral (SCP)	0.0001%	0.999893	0.318503	3.139349	0.1822
円筒	Angle-Based (ABF)	0.0021%	0.999893	0.318504	3.139340	0.0243
半球	Spectral (SCP)	13.4943%	0.999893	0.770752	1.297295	0.0185
半球	Angle-Based (ABF)	10.4452%	0.999893	0.889094	1.124620	0.0165

わかったこと

- ・展開できる形は、ゆがみ0で開く。円筒の縦横比は式と6桁一致。
- ・球はどう開いてもゆがむ。半球で 10～13%。切れ目を増やすほど減るはず（今回は試していない）。
- ・UV は幅1に収められる。そのままでは、部品ごとのテクセル密度はそろわない。

次にやること

- ・半球に切れ目を足していくと、ゆがみがどこまで減るか
- ・ Preserve Seams を入れて、既存の切れ目を生かしたときの結果
- ・ uvunwrap / uvproject との、ゆがみと速さの比較

relax は点を $2 \times \text{pscale}$ 近くまで離す — 余裕があれば97%、詰め込みに近いと73~77%

2026-09-24

何を作ったか

1×1 の板に点を散らして pscale を持たせ、relax (Point Relax) にかけた。 pscale が半径なら、重なりが解けたとき点どうしは $2 \times \text{pscale}$ 以上離れるはず。板に六角形で詰めたときの間隔 $\sqrt{2/(\sqrt{3} \cdot n)}$ を基準にして、 pscale をその $0.25 \cdot 0.4 \cdot 0.5$ 倍にした (0.5 倍だと、ちょうど六角形に詰めたときに触れ合う大きさ)。

relax の前は、500 点で最も近い2点の距離が 0.0009 (ほぼ重なっている)。

余裕があるとき (0.25 倍) は、最小距離が $2 \times \text{pscale}$ の 97~98% まで届いた (50回)。 pscale は半径として働いている。ぴったり $2 \times \text{pscale}$ にはならず、少し食い込む。

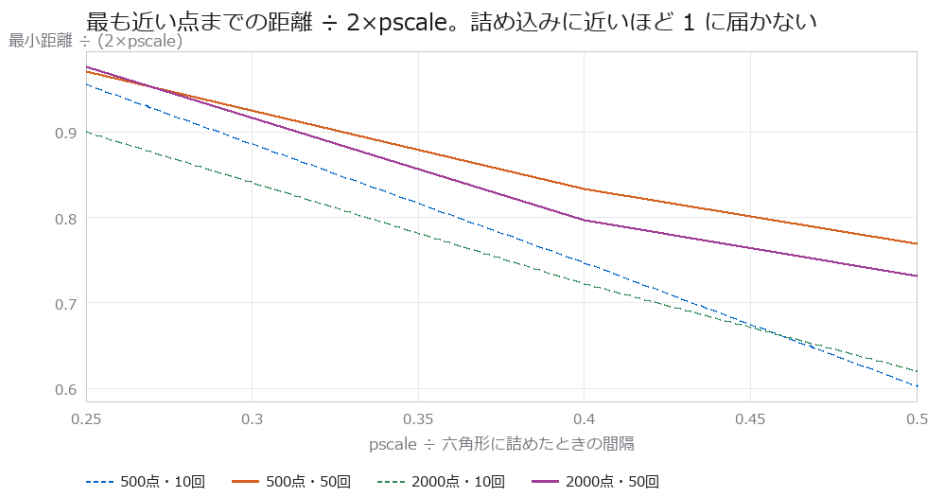
詰め込みに近い (0.5 倍) と、届かない。 50回で 73~77%。回数を10→50に増やすと少し伸びる (500点で 60%→77%) が、1には届かなかった。

板の外にはみ出した点は、どの場合も最大 4 個 (2000点中)。2つめの入力に板を渡して、面の上に留めた。

速さ: 2000点・50回で約0.1秒。

点の数・pscale・回数と、最も近い点までの距離

最小距離は、点ごとに `nearpoints` で一番近い点を探して、その距離の最小。scatter の Relax Iterations は切った。



pscale が小さいほど $2 \times \text{pscale}$ に近づく。破線は10回、実線は50回。

点	pscale	基準比	回数	最小距離	平均距離	最小 ÷ $2 \times \text{pscale}$	はみ出し	秒
500	0.01201	0.25	10	0.02298	0.02793	0.956	0	0.013
500	0.01201	0.25	50	0.02333	0.02795	0.971	0	0.013
500	0.01922	0.4	10	0.02874	0.03765	0.748	0	0.015
500	0.01922	0.4	50	0.03205	0.03886	0.834	0	0.019

点	pscale	基準比	回数	最小距離	平均距離	最小 ÷ 2pscale	はみ出し	秒
500	0.02403	0.5	10	0.02899	0.04173	0.603	0	0.016
500	0.02403	0.5	50	0.03696	0.04441	0.769	0	0.022
2000	0.00601	0.25	10	0.01082	0.01390	0.901	1	0.023
2000	0.00601	0.25	50	0.01173	0.01392	0.976	4	0.063
2000	0.00961	0.4	10	0.01390	0.01880	0.723	1	0.028
2000	0.00961	0.4	50	0.01532	0.01942	0.797	1	0.086
2000	0.01201	0.5	10	0.01492	0.02074	0.621	0	0.022
2000	0.01201	0.5	50	0.01758	0.02211	0.732	1	0.103

わかったこと

- ・ pscale は半径。余裕があれば、点どうしは $2 \times \text{pscale}$ の約97%まで離れる。
- ・ 詰め込みに近いと届かない。板に入る限界の半分の pscale（六角形で触れ合う大きさ）では 73～77%。回数を増やしても大きくは変わらない。
- ・ 面の上に留めるなら、2つめの入力に面を渡す。はみ出しは 2000点中 最大4点。

次にやること

- ・ Relax in 3D Space を入れたとき、面から浮くか
- ・ scatter の Relax Iterations と、relax を別に通すのとの違い
- ・ 曲がった面（球）の上で、同じ距離が保てるか

revolve の面積・体積は k 角の式に6桁一致 — ただし閉じた断面を回すと体積がマイナス（裏返し）

2026-09-24

何を作ったか

線分 (1, 0)→(0.4, 1.5) を y 軸のまわりに一周させ、できた円錐台の側面積を式と比べた。一周を k 分割すると、1枚の面は等脚台形になるので、面積は k 角の角錐台の式で出せる。

****側面積は、5通りすべて k 角の式と一致****（差は最大 0.000000）。なめらかな円錐台の式との比は、k=16 で 0.9910、k=256 で 0.999964。

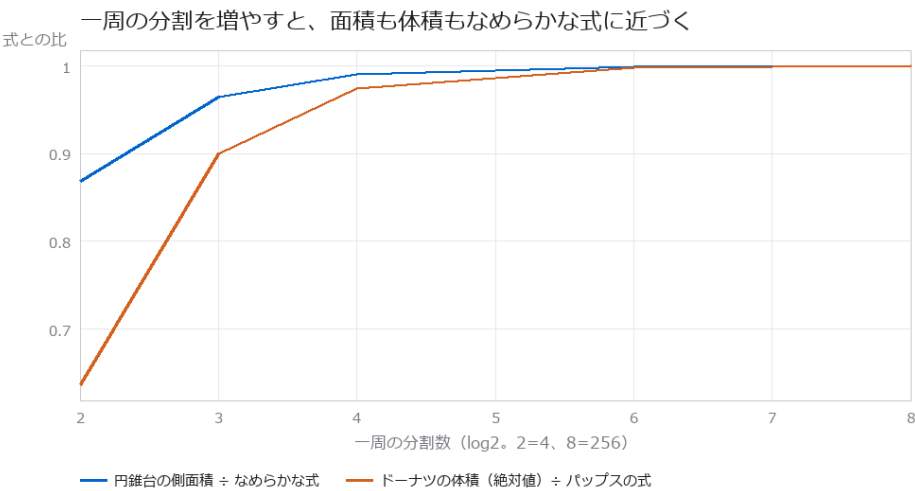
次に、正方形（0.5×0.5、軸から 1.0～1.5）を回して、四角い断面のドーナツを作った。体積はパップス・ギュルダンの定理（断面積 × 重心の回る長さ）で出る。****大きさは k 角の式と一致****（差は最大 0.000000）。****ところが符号がすべてマイナスだった。* k=64 で -1.960343。面の表が内側を向いている（裏返し）ということ。**

直し方は3つとも効いた: revolve の Reverse Cross Sections を入れる (+1.960343)、後ろに reverse を置く (+1.960343)、回す前の断面を reverse で裏返す (+1.960343)。

断面は xy 平面で反時計回り（手前から見て）に点を並べた。時計回りに並べていれば、最初から表向きになるはず（確かめていない）。

円錐台の側面積（下の半径1・上の半径0.4・高さ1.5）

k 角の式 = $k \times (\text{上下の辺の和} / 2) \times \text{斜めの高さ}$ 。上下の辺 = $2r \cdot \sin(n/k)$ 、斜めの高さ = $\sqrt{(h^2 + ((r_2 - r_1) \cdot \cos(n/k))^2)}$ 。



どちらも分割を増やすと比が1に近づく。体積は符号を外して比べた。

一周の分割	面の数	面積	k 角の式	なめらかな式
4	4	6.172714	6.172714	7.105558
8	8	6.854042	6.854042	7.105558
16	16	7.041433	7.041433	7.105558
64	64	7.101525	7.101525	7.105558

一周の分割	面の数	面積	k 角の式	なめらかな式
256	256	7.105305	7.105305	7.105558

四角い断面のドーナツの体積

k 角の式 = $(k/2) \cdot \sin(2\pi/k) \cdot (1.5^2 - 1.0^2) \cdot 0.5$ 。なめらかな式 = $0.25 \times 2\pi \times 1.25$ 。

一周の分割	面の数	体積 (measure)	k 角の式	なめらかな式
4	16	-1.250000	1.250000	1.963495
8	32	-1.767767	1.767767	1.963495
16	64	-1.913417	1.913417	1.963495
64	256	-1.960343	1.960343	1.963495
256	1024	-1.963298	1.963298	1.963495

わかったこと

- ・回した面の面積は k 角の式どおり。既定の分割のまま面積を出すなら、k 角の式で補正できる。
- ・閉じた断面を回すと、裏返しになることがある。体積がマイナスになったら、Reverse Cross Sections を入れる。
- ・裏返しは見た目では気づきにくい。measure の体積の符号で見つかる（実験098 の「答えが計算で出る形で試す」がここでも効いた）。

次にやること

- ・断面を時計回りに並べたときに、最初から表向きになるか
- ・角度の範囲だけ回したとき（End Caps あり）の体積
- ・NURBS で回したときの面積が、なめらかな式に一致するか

面に沿った距離は Surface が正解 — Edge と attribfill は辺の道のり、Heat は約1%短い

2026-09-24

何を作ったか

2×2 の平らな板の真ん中から、面に沿った距離を5通りの方法で測り、本当の距離 $\sqrt{(x^2+z^2)}$ と比べた。網は、四角（41×41）・三角（同じ点を三角形でつないだもの）・remesh（向きがばらばらの三角形）の3つ。

distancealonggeometry の Surface（既定）は、ほぼ正解。 四角の網ですれ 0.000000、三角の網・remesh でも最大 0.0003 未満。辺をたどるのではなく、面の上をまっすぐ進んだ距離を出している。

Edge は、辺をたどった道のり。 四角の網では $|x|+|z|$ になり、角で最大 0.5858（= $2 - \sqrt{2}$ ）ずれる。attribfill の Arrival Time は、3つの網すべてで Edge と違う値だった（実験115の「縦横の道のり」の正体）。remesh した網では辺の向きがばらけるので、ずれの平均は 0.039 まで減る。

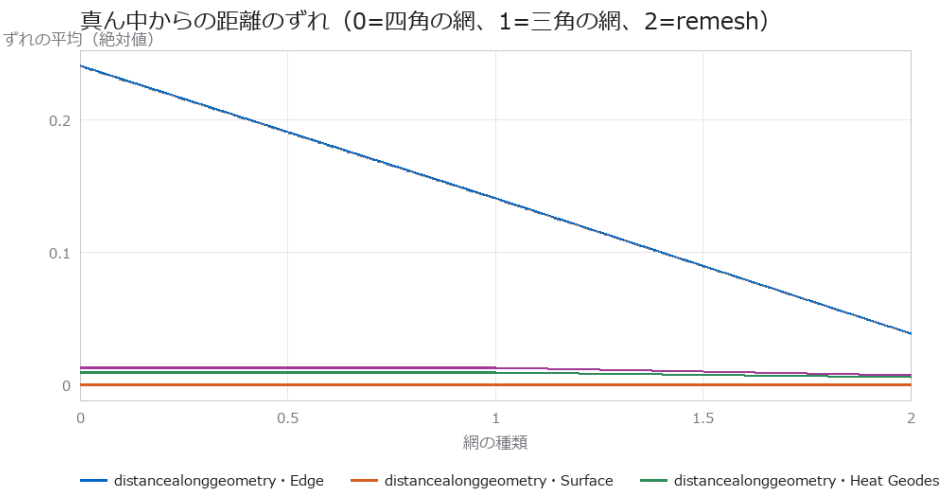
Heat Geodesic は、どの網でも少し短く出る（平均で 0.0090、heatgeodesic ノードは 0.0125）。網の向きには左右されにくい。

速さは、どれも 1,700～1,900 点で 0.04 秒以下（最初の1回を除く）。

網と測り方ごとの、本当の距離とのずれ

始まりは板の真ん中の点。remesh

では原点に一番近い点を始まりにして、全体をずらして原点に合わせた。



Surface（緑）はどの網でも0に近い。Edge と attribfill（破線）は重なる。

網	測り方	ずれの平均	ずれの最大	ずれの向き（平均）	秒
四角の網	distancealonggeometry・Edge	0.240251	0.585786	+0.240251	0.0350
四角の網	distancealonggeometry・Surface	0.000000	0.000000	-0.000000	0.0036

網	測り方	ずれの平均	ずれの最大	ずれの向き（平均）	秒
四角の網	distancealonggeometry・Heat Geodesic	0.008959	0.046040	-0.008959	0.1304
四角の網	heatgeodesic	0.012460	0.058876	-0.012452	0.0203
四角の網	attribfill・Arrival Time	0.240251	0.585786	+0.240251	0.0124
三角の網	distancealonggeometry・Edge	0.140238	0.585786	+0.140238	0.0066
三角の網	distancealonggeometry・Surface	0.000005	0.000269	+0.000005	0.0043
三角の網	distancealonggeometry・Heat Geodesic	0.008959	0.046040	-0.008959	0.0165
三角の網	heatgeodesic	0.012460	0.058876	-0.012452	0.0136
三角の網	attribfill・Arrival Time	0.140238	0.585786	+0.140238	0.0020
remesh した網	distancealonggeometry・Edge	0.038608	0.080720	+0.038608	0.0075
remesh した網	distancealonggeometry・Surface	0.000018	0.000235	+0.000018	0.0048
remesh した網	distancealonggeometry・Heat Geodesic	0.005626	0.017211	-0.005625	0.0223
remesh した網	heatgeodesic	0.007247	0.021913	-0.007245	0.0188
remesh した網	attribfill・Arrival Time	0.038607	0.080720	+0.038607	0.0023

わかったこと

- ・面に沿った本当の距離がほしいなら、distancealonggeometry の Surface。既定のまま使える。
- ・Edge と attribfill の Arrival Time は、辺の道のり。網の向きに引っぱられる。四角の網では最大で約 41% ($2 - \sqrt{2}$ を $\sqrt{2}$ で割った割合) 長くなる。
- ・Heat Geodesic は約1%短い。向きには強いが、ぴったりではない。
- ・remesh すると辺の道のりのずれは小さくなるが、0 にはならない。

次にやること

- ・曲がった面（球・波打つ面）で、Surface がどこまで正確か
- ・findshortestpath の cost が Edge と同じになるか
- ・Heat Geodesic の Smoothing を変えると、短く出る分が変わるか

measurethickness は殻の厚みを6桁で返す ー 板の縁ではぼかしで細く出る。box の Use Divisions は面にならない

2026-09-24

何を作ったか

厚みが分かっている形を3つ作って、measurethickness の値と比べた。

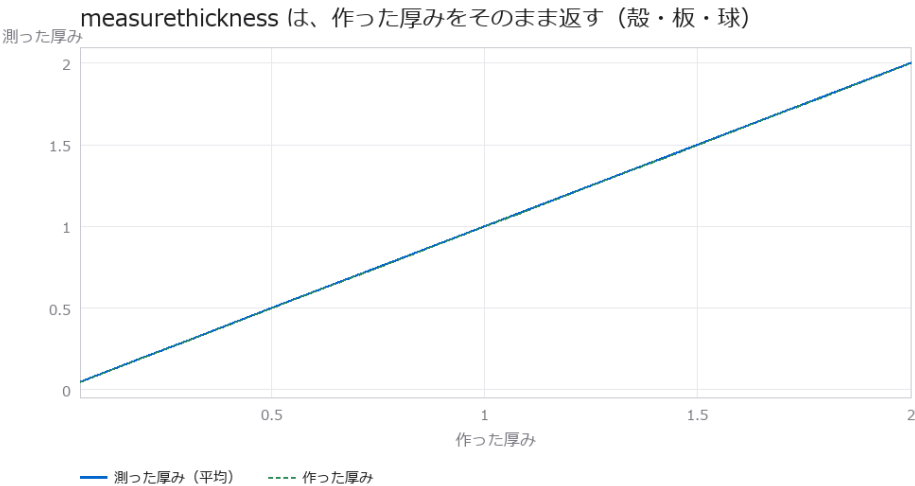
**殻（半径1の球の中に、裏返した半径 r の球）は、ほぼぴったり $1-r$ 。 ** $r = 0.5 \cdot 0.8 \cdot 0.95$ のすべての点で、ずれは最大 0.000001。詰まった球は直径 2 に対して 1.999994~2.000000。

**板（ $4 \times 0.3 \times 4$ ）は、既定のままだと少し薄く出る。 ** 上と下の面の真ん中あたりの点で、平均 0.2967、最小 0.2799。ぼかし（Use Blur）と、まわりの中央値（Median of Neighbors）を切ると、全部 0.300000 になった。縁に近い所の短い値が、ぼかしで内側へにじんでいる。

**つまずいた点: box を Polygon のまま Use Divisions で割ると、面ではなく開いた線の籠になる。 ** 616 本すべてが閉じていない線で、体積は 0。measurethickness は 0.002308 という意味の無い値を返した。点を増やした箱がほしいときは、Primitive Type を Polygon Mesh にして Axis Divisions で割る。

作った厚みと、測った厚み

球は polymesh 96×192 。板は上と下の面の、縁から 0.5 以上内側の点だけを数えた。



殻・詰まった球は線の上。板は既定のぼかしで少し下に出る。

形	作った厚み	平均	最小	最大	点の数
板 0.3 (Polygon Mesh)	0.3	0.296737	0.279922	0.300000	450
板 0.3 (Polygon + Use Divisions)	0.3	0.002308	0.002308	0.002308	512
殻 1-0.5	0.5	0.500000	0.499999	0.500001	36100
殻 1-0.8	0.2	0.200000	0.199999	0.200001	36100

形	作った厚み	平均	最小	最大	点の数
殻 1-0.95	0.05	0.050000	0.050000	0.050000	36100
詰まった球（直径 2）	2	1.999996	1.999994	2.000000	18050

板で、ぼかしを切ったとき

Use Blur と Median of Neighbors を切った。

作り方	既定の平均	既定の最小	ぼかし無しの平均	ぼかし無しの最小
板 0.3（Polygon Mesh）	0.296737	0.279922	0.300000	0.300000
板 0.3（Polygon + Use Divisions）	0.002308	0.002308	0.002308	0.002308

わかったこと

- ・丸い殻の厚みは正確。ずれは 0.000001 程度。
- ・角のある形では、ぼかしで縁の値がにじむ。正確な値がほしいなら Use Blur と Median of Neighbors を切る。
- ・box の Use Divisions（Polygon のとき）は開いた線の籠。measure の体積が0になる。点を増やすなら Polygon Mesh にする。
- ・速い。36,100 点の殻で約0.1秒。

次にやること

- ・Max Fitting Sphere と Ray Length で、角の近くの値がどう違うか
- ・Use SDF を入れたときの速さと精度
- ・厚みが場所で変わる形（くさび）で、変化を正しく追えるか